I am doing a lab on R, on positron using tidyverse and on the topic of 3D mapping and rasterisation.

Working with 3D mapping and rasterisation in Positron is highly efficient when combining the `tidyverse` with modern spatial packages. The standard workflow for this involves integrating specific raster manipulation tools with 3D rendering engines to transform your data into a visual topography.

Spatial Package Stack

Package	Primary Function	Tidyverse Integration
terra	Processing and rasterising spatial data sets [1].	Base package; requires extensions for tidy syntax [2].
tidyterra	Providing `dplyr` verbs and `ggplot2` geometries for spatial data [2].	Native integration with `tidyverse` workflows [1].
rayshader	Building 3D models via raytracing and hillshading algorithms [3].	Converts `ggplot2` outputs directly into 3D via `plot_gg()` [4].

Rasterisation and Manipulation

The `terra` package is currently the premier tool for handling raster data in R, significantly outperforming older standards like the `raster` package. To bridge this processing power with your `tidyverse` workflow, `tidyterra` acts as the vital connector. It allows you to use standard data manipulation verbs on raster objects and provides custom geometries to plot raster layers natively in `ggplot2`. [1] [2]

3D Map Generation

For generating three-dimensional visualizations, `rayshader` handles the heavy lifting by converting elevation matrices into detailed scenes. You can easily transition from a standard 2D map to a 3D visualization by passing your `ggplot2` output directly into the package's rendering functions. Once rendered, you can further manipulate camera angles, light directions, and rendering quality to produce high-resolution 3D maps. [3] [4]

What specific type of spatial data are you attempting to rasterise and map in 3D for your lab?

Could you deeply explain how anom and sst work? When do I use one over the other?

SST is the **actual** sea-surface temperature field, while ANOM is the departure of that field from a climatological average for the same place and time of year. Use SST when absolute temperature matters, and use ANOM when you want to see whether conditions are unusually warm or cool relative to normal. [16] [17] [18] [19]

What SST is

SST stands for sea surface temperature, and SST maps show the analyzed or observed temperature of the ocean surface itself rather than a comparison against normal conditions. In operational products, those values are commonly built from satellite and in-water observations and then interpolated onto a regular grid so gaps can be filled consistently across space and time. Depending on the product, the "surface" may be defined slightly differently; for example, Australia's Bureau of Meteorology describes its SST analysis as the temperature of the uppermost 10 metres under well-mixed conditions. [18] [19] [16]

What ANOM is

ANOM means anomaly, which is simply the difference between the current value and a long-term mean for that same location and date or month. One tutorial defines it explicitly as HIST minus NORM, where HIST is the historical actual value and NORM is the climatological normal. A positive anomaly means the sea surface is warmer than average, while a negative anomaly means it is cooler than average. [17] [16]

$$\text{ANOM} = \text{Observed SST} - \text{Climatological Mean SST}$$

The exact baseline used for the climatological mean depends on the dataset; examples in operational products include 1981–2010 and 1971–2000. That means an anomaly value is always relative, so 1 degree above normal in one dataset may not match 1 degree above normal in another if the baselines differ. [16] [18]

How they behave in maps

SST maps preserve the geographic temperature pattern you would physically measure, so tropical waters look warm and high-latitude waters look cool even when both are perfectly normal for the season. ANOM maps remove most of that background seasonal pattern, which makes unusual warm pools, cold tongues, marine heatwaves, or El Niño/La Niña-type signals much easier to see. This is why anomaly maps are often better for pattern detection, while SST maps are better for interpreting the real thermal environment. [20] [19] [21] [17] [18]

When to use each

Use SST when your question depends on absolute thresholds, such as whether water is warm enough to support coral bleaching risk analysis, cyclone intensification, or species habitat limits. Use ANOM when your question is “what is unusual right now,” especially if you are comparing different regions, months, or years with very different normal temperatures. In practice, ANOM is usually the better choice for climate diagnostics and change detection, while SST is usually the better choice for environmental conditions and direct physical interpretation.[22] [19] [17] [18]

In your R lab

If you are rasterising or making 3D surfaces in R, plotting SST will make the surface reflect real temperature magnitudes, whereas plotting ANOM will emphasize departures from normal and often produce a clearer centered color scale around zero. A good rule is: choose SST when the legend should answer “how warm is it,” and choose ANOM when the legend should answer “how unusual is it.” If your lab compares seasons or years, ANOM is usually easier to interpret because it controls for the normal seasonal cycle that would otherwise dominate the map.[19] [17] [16]

Would you like me to explain this again in the context of `tidyverse` code, for example how you would compute ANOM from an SST raster in R?

⁂

For a little context, we are working with climate maps.

When to use diverging and when to use sequential scales?

Use a **sequential** scale when your climate variable runs from low to high with no meaningful center, and use a diverging scale when the middle value matters, such as zero, the long-term average, or a threshold. In climate maps, that usually means SST uses sequential colors, while SST anomaly uses diverging colors because “normal” is the reference point.[31] [32] [33] [34]

Sequential scales

Sequential scales are for ordered data where the main task is to show progression from lower values to higher values. They work best when the reference level is effectively the minimum or low end of the range, not a middle value. They are also more intuitive for readers because darker or more saturated colors naturally read as “more.”[35] [32] [31]

Diverging scales

Diverging scales are for data with a meaningful midpoint, and they use a neutral color near that center with one hue for values below it and another hue for values above it. Good midpoints include zero, the average, the median, a climatology, or a policy threshold. They also

emphasize both extremes more strongly than sequential scales, which is useful when the story is about unusually low and unusually high conditions. [32] [31]

Climate map examples

Here is the practical rule for common climate layers:

Variable	Best scale	Why
SST	Sequential [34] [32]	It shows actual temperature from cooler to warmer, so absolute magnitude matters more than deviation from a center. [34] [32]
SST anomaly	Diverging [33] [31]	Zero is meaningful because it separates cooler-than-normal from warmer-than-normal water. [33] [31]
Total rainfall	Sequential [32]	The variable usually runs from low totals to high totals without a natural middle reference. [32]
Rainfall anomaly	Diverging [31] [32]	The important question is whether conditions are below normal or above normal. [31] [32]

How to decide

Ask yourself: “Do I want to show how much, or do I want to show how unusual?” If the answer is “how much,” use sequential; if the answer is “how unusual relative to normal,” use diverging. Also note that diverging scales are less intuitive on their own, so they need a clear legend and carefully chosen colors to show which side means above and below the reference. [33] [31] [32]

In your lab

For climate raster maps, sequential scales are usually the safer choice for raw fields such as SST, precipitation amount, or elevation-like surfaces because those are magnitude maps. Diverging scales are usually the better choice for anomaly rasters, trend rasters, or standardized departures because the center value carries the meaning. A simple shortcut is: raw climate field equals sequential, anomaly field equals diverging. [34] [36] [31] [32] [33]

Are your lab maps showing raw SST, anomalies, or both?

⁂

In the lab sheet, these are the packages that we are allowed/told to use:

3.2 Start Coding

```
#| label: setup
#| message: false
library(tidyverse)
library(stars)
library(scales)
```

```
library(ggnewscale)
library(units)
```

The error that I got:
install.packages("stars")
Installing package into ‘/home/aiman/R/x86_64-pc-linux-gnu-library/4.5’
(as ‘lib’ is unspecified)
Warning: unable to access index for repository https://cloud.r-project.org/src/contrib:
cannot open URL 'https://cloud.r-project.org/src/contrib/PACKAGES'
Warning message:
package ‘stars’ is not available for this version of R

A version of this package for your version of R might be available elsewhere,
see the ideas at
https://cran.r-project.org/doc/manuals/r-patched/R-admin.html#Installing-packages

Your error most likely is **not** that `stars` is incompatible with R 4.5; the more important line is that R could not open the CRAN package index at `https://cloud.r-project.org/src/contrib/PACKAGES`. Because R could not read the repository index, it then falls back to the generic message saying the package is “not available for this version of R,” which can be misleading. [44] [45]

What the error means

The `stars` package does exist and is published for current R setups, including a CRAN release and an r-universe release. The package is for reading, manipulating, writing, and plotting spatiotemporal raster and vector data cubes, so it is exactly the kind of package your climate-mapping lab would use. [45] [46] [44]

Most likely cause

The first warning says R was unable to access the CRAN repository index, which usually points to a connectivity, repository, SSL, proxy, or environment restriction problem rather than a missing package. Since R never successfully read the package list, it could not confirm availability from CRAN. In other words, the “not available” line is probably a downstream symptom, not the root problem. [44] [45]

What to try

Try this install command first, because the `stars` project publishes an r-universe repository alongside CRAN: [44]

```
install.packages(
  "stars",
  repos = c("https://r-spatial.r-universe.dev", "https://cloud.r-project.org")
)
```

If that still fails with the same “cannot open URL” warning, the issue is probably your machine or lab environment blocking package downloads rather than the package itself. In that case, your instructor may expect you to use a preconfigured environment where `stars` is already installed. [45] [44]

Another possible issue

`stars` relies on spatial infrastructure such as GDAL through related packages, and installation problems for this stack can also come from missing system dependencies. A Posit Community thread about failed `stars` installation specifically mentions missing `pkg-config` and `gdal-config`, which shows that spatial package installs can fail even after repository access works. [47] [44]

What to tell your instructor

A precise way to report it is: “R cannot access the CRAN repository index, so `install.packages("stars")` fails before it can resolve the package.” That is more accurate than saying “`stars` does not support my R version,” because public package listings show `stars` is available. [45] [44]

Use this exact wording if you want:

```
I tried to install `stars`, but R could not access the CRAN package index (`cannot o
```

Paste the next error you get after trying the r-universe install command, and I’ll help you interpret that line by line.

⁂

installing to /home/aiman/R/x86_64-pc-linux-gnu-library/4.5/00LOCK-s2/00new/s2/libs

** R
** data
*** moving datasets to lazyload DB
** inst
** byte-compile and prepare package for lazy loading
** help
*** installing help indices
*** copying figures
** building package indices
** testing if installed package can be loaded from temporary location
** checking absolute paths in shared objects and dynamic libraries
** testing if installed package can be loaded from final location
** testing if installed package keeps a record of temporary installation path

- DONE (s2)
  ERROR: dependency ‘units’ is not available for package ‘sf’
  Perhaps try a variation of:
  install.packages('units')
- removing ‘/home/aiman/R/x86_64-pc-linux-gnu-library/4.5/sf’
  ERROR: dependencies ‘sf’, ‘units’ are not available for package ‘stars’
  Perhaps try a variation of:
  install.packages(c('sf', 'units'))
- removing ‘/home/aiman/R/x86_64-pc-linux-gnu-library/4.5/stars’

The downloaded source packages are in
‘/tmp/Rtmp3v6BnU/downloaded_packages’
Warning messages:
1: In install.packages("stars", repos = c("https://r-spatial.r-universe.dev", :
installation of package ‘units’ had non-zero exit status
2: In install.packages("stars", repos = c("https://r-spatial.r-universe.dev", :
installation of package ‘sf’ had non-zero exit status
3: In install.packages("stars", repos = c("https://r-spatial.r-universe.dev", :
installation of package ‘stars’ had non-zero exit status

I then tried again:
install.packages("stars", repos = "https://r-spatial.r-universe.dev")
Installing package into ‘/home/aiman/R/x86_64-pc-linux-gnu-library/4.5’
(as ‘lib’ is unspecified)
Warning: dependency ‘units’ is not available
also installing the dependency ‘sf’

trying URL 'https://r-spatial.r-universe.dev/src/contrib/sf_1.1-2.tar.gz?sha256=fba8d73a0dccb383e2e73fe226fb1829651d0c27525f385fc99ec7a135e2acde&file=/sf_1.1-2.tar.gz'
trying URL 'https://r-spatial.r-universe.dev/src/contrib/stars_0.7-3.tar.gz?sha256=ab0aae1b192a6cc30e4ec533dab4e6c797542f7c4f19223edfa28493d1676d13&file=/stars_0.7-3.tar.gz'
ERROR: dependency ‘units’ is not available for package ‘sf’
Perhaps try a variation of:
install.packages('units')

- removing ‘/home/aiman/R/x86_64-pc-linux-gnu-library/4.5/sf’
  ERROR: dependencies ‘sf’, ‘units’ are not available for package ‘stars’
  Perhaps try a variation of:
  install.packages(c('sf', 'units'))
- removing ‘/home/aiman/R/x86_64-pc-linux-gnu-library/4.5/stars’

The downloaded source packages are in
‘/tmp/Rtmp3v6BnU/downloaded_packages’
Warning messages:
1: In install.packages("stars", repos = "https://r-spatial.r-universe.dev") :
installation of package ‘sf’ had non-zero exit status

2: In install.packages("stars", repos = "https://r-spatial.r-universe.dev") :
installation of package ‘stars’ had non-zero exit status

Yes — this narrows the problem down a lot. The root failure is now `units`, and because `sf` depends on `units`, and `stars` depends on both `sf` and `units`, the whole install chain stops there. [59] [60] [61]

What is failing

Your log shows that `s2` installed successfully, so the current blocker is no longer repository access. The next failure is `ERROR: dependency 'units' is not available for package 'sf'`, and that in turn causes `stars` to fail because it needs `sf` and `units`. [60] [61] [59]

Likely cause

A very common reason `units` fails on Linux is that the system library for UDUNITS is missing, specifically `libudunits2`. A documented example of the same error chain shows `units` failing with a configure error saying `libudunits2.so was not found`, and recommends installing `libudunits2-dev` on Debian or Ubuntu systems. Since `sf` is part of the spatial stack that binds to external libraries such as GDAL, GEOS, and PROJ, these installs often depend on system packages outside R itself. [62] [61] [60]

What to do next

Run this first by itself, because it should expose the real low-level error for `units` instead of hiding it behind `stars`:

```
install.packages("units")
```

If the output mentions `libudunits2.so was not found`, then the fix is at the system level, not inside R. On Ubuntu or Debian, the usual fix is: [61]

```
sudo apt install libudunits2-dev
```

That package is specifically recommended in a reported Linux case with the same `units` installation failure. [61]

If this is a lab machine

If you do not have admin rights, you probably will not be able to fix this yourself because the missing dependency is a Linux system library, not just an R package. In that case, ask your instructor or lab admin to install `libudunits2-dev`, and possibly confirm that the full spatial stack needed by `sf` is present as well. [62] [60] [61]

What to send your instructor

You can send this:

```
`stars` is failing because `sf` cannot be installed, and `sf` is failing because the
```

That summary matches the dependency chain shown by your log and the documented Linux fix for `units`. [59] [61]

Paste the output of `install.packages("units")` next, especially the lines near `configure: error`, and I'll decode it line by line.

⁂

Could you assist with this task,

The input data are divided into two folders, one for each December.
Each folder contains one NetCDF file for every day of that month:

oisst_daily/
|-- el_nino_2009-12/
| |-- oisst-avhrr-v02r01.20091201.nc
| |-- ...
| `-- oisst-avhrr-v02r01.20091231.nc -- la_nina_2010-12/
|-- oisst-avhrr-v02r01.20101201.nc
|-- ...
`-- oisst-avhrr-v02r01.20101231.nc
Apply read_stars() to create two stars objects:

dec_2009, containing the 31 daily grids for December 2009
dec_2010, containing the 31 daily grids for December 2010
Each completed object should contain exactly two attributes (anom and ice) and three dimensions:

Yes. For this task, the key idea is that `read_stars()` can take a **vector of file paths** and combine them into one stars object, and when multiple files are read, they can be merged along a named dimension such as `time`. Since your folders contain one NetCDF file per day, you want to build a sorted list of the 31 files in each folder and read only the `anom` and `ice` variables into one object per month. [74] [75] [76]

What the object should look like

Your final `dec_2009` and `dec_2010` objects should each have two attributes, `anom` and `ice`, and three dimensions that are effectively longitude, latitude, and time. The `stars` documentation explains that NetCDF files commonly expose dimensions such as `lon`, `lat`, and `time`, and that `read_stars()` can combine multiple files along a dimension you specify. [75] [76]

Code structure

Use `list.files()` to collect the NetCDF filenames, make sure they are in order, and then pass that character vector into `read_stars()`. Because you only want two attributes, set `sub = c("anom", "ice")`, and because each file is one day, set `along = "time"` so the files stack across time instead of becoming separate attributes. [74] [75]

```
library(tidyverse)
library(stars)
library(scales)
library(ggnewscale)
library(units)

files_2009 <- list.files(
  "oisst_daily/el_nino_2009-12",
  pattern = "\\.nc$",
  full.names = TRUE
) |> sort()

files_2010 <- list.files(
  "oisst_daily/la_nina_2010-12",
  pattern = "\\.nc$",
  full.names = TRUE
) |> sort()

dec_2009 <- read_stars(
  files_2009,
  sub = c("anom", "ice"),
  along = "time"
)

dec_2010 <- read_stars(
  files_2010,
  sub = c("anom", "ice"),
  along = "time"
)
```

This matches the documented behavior that when `.x` contains multiple files, they are read and combined, and `along` controls the dimension used for combining them. [75]

How to check it worked

After reading, inspect the objects with `print(dec_2009)`, `st_dimensions(dec_2009)`, and `names(dec_2009)`. You should see exactly the two attributes `anom` and `ice`, and the dimensions should include spatial axes plus a `time` dimension of length 31 if the files were combined correctly. [76] [75]

```
names(dec_2009)
st_dimensions(dec_2009)

names(dec_2010)
st_dimensions(dec_2010)
```

If the files are daily OISST NetCDFs, the dimensions may print as `lon`, `lat`, and `time`, and sometimes there may also be a singleton extra dimension such as `zlev` depending on how the source file is structured. If your lab sheet says the completed object must have exactly three dimensions, you may need to drop a size-1 dimension after reading if one appears. [76]

If a fourth dimension appears

Some OISST NetCDF examples in `stars` show dimensions `lon`, `lat`, `zlev`, and `time`, where `zlev` has length 1. If that happens in your files, subset that singleton dimension away so the result has exactly three dimensions while keeping the two attributes. [75] [76]

A common pattern is:

```
dec_2009 <- dec_2009[,,,1,, drop = TRUE]
dec_2010 <- dec_2010[,,,1,, drop = TRUE]
```

But the exact indexing depends on the dimension order shown by `st_dimensions(dec_2009)`, so check that output first before subsetting. [76] [75]

Safer alternative

Because these are NetCDF files, `read_ncdf()` is also a natural fit for reading selected NetCDF variables directly, and the documentation explicitly supports `var = c("anom", "ice")`. However, since your task specifically says to use `read_stars()`, the multi-file `read_stars(..., sub = c("anom","ice"), along = "time")` approach is the one most aligned with the lab instructions. [77] [75]

Paste the output of `st_dimensions(dec_2009)` once you run it, and I can tell you whether you need to remove a singleton dimension.

⁂

This is what I did instead:

3.3 Import the Daily Grids as `stars` Data Cubes

```
# 1. Get lists of file paths for each folder
files_2009 <- list.files("oisst_daily/el_nino_2009-12", pattern = "\\.nc$", full.name
files_2010 <- list.files("oisst_daily/la_nina_2010-12", pattern = "\\.nc$", full.name

# 2. Read and bind the files along the time dimension
dec_2009 <- read_stars(files_2009, quiet = TRUE) %>%
  select(anom, ice)

dec_2010 <- read_stars(files_2010, quiet = TRUE) %>%
  select(anom, ice)
```

But i am running into another issue:
Why is this not working?

Error:
! file not found
Backtrace:
■

1. └─stars::read_stars("oisst_daily/el_nino_2009-12/", proxy = FALSE)

dec_2009 <- read_stars("oisst_daily/el_nino_2009-12/", proxy = FALSE)

Yes — the immediate reason for that error is that you passed a **folder path** to `read_stars()`, but the function is documented and exemplified with a file path or a character vector of file paths, not a directory name. So `read_stars("oisst_daily/el_nino_2009-12/")` fails because that string points to a folder, not to a NetCDF file the function can open. [91] [92]

Why your first approach is closer

Your `list.files(..., full.names = TRUE)` step is the right direction because it creates a character vector of `.nc` file paths, which matches how `read_stars()` is shown being used with multiple inputs. The examples for `read_stars()` show that when you pass multiple files, you can control how they are combined by using `along = ...`, which is the key part for building a time dimension. [93] [91]

What to change

For your lab task, use the file vector, not the folder name, and add `along = "time"` so the 31 daily files are stacked into one cube along time. Since you only need `anom` and `ice`, reading just those attributes during import is the cleanest approach for this task. [94] [93] [91]

```
files_2009 <- list.files(
  "oisst_daily/el_nino_2009-12",
  pattern = "\\.nc$",
  full.names = TRUE
) |> sort()

files_2010 <- list.files(
  "oisst_daily/la_nina_2010-12",
  pattern = "\\.nc$",
  full.names = TRUE
) |> sort()

dec_2009 <- read_stars(
  files_2009,
  along = "time",
  sub = c("anom", "ice"),
  quiet = TRUE
)

dec_2010 <- read_stars(
  files_2010,
```

```
  along = "time",
  sub = c("anom", "ice"),
  quiet = TRUE
)
```

The exact mistake

This line is the one that is wrong for the error you showed: [92] [91]

```
dec_2009 <- read_stars("oisst_daily/el_nino_2009-12/", proxy = FALSE)
```

It should be either a single `.nc` file, like `"...20091201.nc"`, or the `files_2009` vector created by `list.files()`. If your goal is one stars object for the whole month, then `files_2009` with `along = "time"` is the version that matches the task best. [93] [91] [92]

One more check

After reading, inspect `names(dec_2009)` and `st_dimensions(dec_2009)` to confirm that you got exactly the attributes `anom` and `ice` and a time dimension representing the daily grids. If you paste that output, I can check whether your object now matches the lab requirement exactly. [95] [94]

⁂

what does the sub = c(...) in the read_stars() function do?

`sub = c(...)` tells `read_stars()` which **sub-datasets** to read from the file, instead of reading everything available. In the documentation, `sub` is defined as a character, integer, or logical value that specifies the name, index, or indicator of the sub-dataset(s) to be read. [108] [109]

In your case

When you write `sub = c("anom", "ice")`, you are telling `read_stars()` to import only the NetCDF components named `anom` and `ice`. For your OISST files, those names correspond to the variables or layers you want in the resulting stars object. [109] [108]

Why use it

Using `sub` is cleaner than reading all variables first and filtering afterward, because it restricts the import at the moment the file is read. That means your resulting object should already contain just the attributes you asked for, which matches your lab requirement more directly. [108] [109]

What the `c(...)` part means

The `c(...)` is just base R syntax for "combine these into a vector." So `c("anom", "ice")` creates a character vector of two names, and `read_stars()` interprets those names as the sub-datasets to load.[108]

Small examples

These are the three common patterns:[108]

```
read_stars("file.nc")
```

This reads the file without explicitly naming sub-datasets.[109] [108]

```
read_stars("file.nc", sub = "anom")
```

This reads only the `anom` sub-dataset.[108]

```
read_stars("file.nc", sub = c("anom", "ice"))
```

This reads only `anom` and `ice`.[108]

For your lab, `sub = c("anom", "ice")` is the right idea because the task explicitly says each completed object should contain exactly those two attributes.

⁂

This version was able to work:

3.3 Import the Daily Grids as `stars` Data Cubes

```
#| label: import-stars
files_2009 <- list.files("C:/Users/mjaba/Documents/GITHUB/CSC3107-InformationVisualis
files_2010 <- list.files("C:/Users/mjaba/Documents/GITHUB/CSC3107-InformationVisualis

dec_2009 <- read_stars(files_2009, along = "time")
dec_2010 <- read_stars(files_2010, along = "time")

dec_2009
```

Yes — this version works because you are now passing `read_stars()` a vector of actual `.nc` file paths, and `along = "time"` tells it to stack those files into one data cube along a time dimension rather than treating them as unrelated inputs.[118] [119]

Why this worked

Your earlier failing version passed a folder path, but this working version first uses `list.files(..., full.names = TRUE)` to generate the individual NetCDF filenames. Then `read_stars(files_2009, along = "time")` combines the daily grids into a single `stars` object across time.[119] [118]

One improvement

Add `sort()` around both file lists so the files are guaranteed to be read in calendar order. Without sorting, you are usually fine, but it is safer not to rely on the filesystem order.

```
files_2009 <- list.files(
  "C:/Users/mjaba/Documents/GITHUB/CSC3107-InformationVisualisation-Orchid/team-orch
  pattern = "\\.nc$",
  full.names = TRUE
) |> sort()

files_2010 <- list.files(
  "C:/Users/mjaba/Documents/GITHUB/CSC3107-InformationVisualisation-Orchid/team-orch
  pattern = "\\.nc$",
  full.names = TRUE
) |> sort()
```

Another improvement

If your lab requires exactly the two attributes `anom` and `ice`, then it is better to request them during import. That keeps the object aligned with the task immediately.[120] [121]

```
dec_2009 <- read_stars(files_2009, along = "time", sub = c("anom", "ice"))
dec_2010 <- read_stars(files_2010, along = "time", sub = c("anom", "ice"))
```

What to check next

Run these after import:

```
names(dec_2009)
st_dimensions(dec_2009)

names(dec_2010)
st_dimensions(dec_2010)
```

You want to confirm that each object has the right attributes and that the time dimension has 31 slices for the 31 days in December.[118] [119]

One likely reason the old version failed

This version also uses a full absolute path, so it avoids any working-directory mismatch. In other words, part of the earlier problem may have been not just the folder-vs-file issue, but also that R may not have been looking in the directory you expected.

Paste the output of `names(dec_2009)` and `st_dimensions(dec_2009)`.

⁂

This is the next task that I am working on which I need help with.

The below is the code that I tried to do for the 2009 mean anom:
mean_anom_2009 <- st_apply(dec_2009["anom"], c("x", "y"), mean, na.rm = TRUE)
mean_ice_2009 <- st_apply(dec_2009["ice"], c("x", "y"), function(x) {
mean(ifelse(is.na(x), 0, x))
})

summary_2009 <- c(mean_anom = mean_anom_2009, mean_ice = mean_ice_2009) |>
mutate(
land = is.nan(mean_anom) & mean_ice == 0,
mean_ice = if_else(land, NA_real_, mean_ice)
) |>
select(land, mean_anom, mean_ice)

I have to do it for 2010 as well, however i would just like to check if what I am doing is correct so far.

Yes — your approach is **basically correct**, and it matches the task description quite closely. The lab sheet says to collapse the `time` dimension with separate `st_apply()` calls for `anom` and `ice`, then combine them and derive `land` from the two means. [122] [123] [124]

What you did right

Your `mean_anom_2009 <- st_apply(dec_2009["anom"], c("x", "y"), mean, na.rm = TRUE)` is the right idea because `st_apply()` keeps the dimensions named in `MARGIN` and applies the function over the remaining dimension, so with `c("x", "y")` it collapses `time`. That is exactly how the task describes turning 31 daily values per cell into one monthly value. [123] [124] [122]

Your `mean_ice_2009` logic is also aligned with the instructions because the sheet says missing daily `ice` values should be treated as 0 before averaging, not omitted with `na.rm = TRUE`. So using `ifelse(is.na(x), 0, x)` inside the function is conceptually correct. [122]

Small fixes

The main issue is your combination step: `c(mean_anom = mean_anom_2009, mean_ice = mean_ice_2009)` is probably not the right way to combine `stars` objects. The lab sheet explicitly says to combine the two means with `c()`, but in `stars` that usually means combining the objects directly, not assigning names inside base `c()` the way you would with plain vectors.[125] [122]

A safer version is:

```
summary_2009 <- c(mean_anom_2009, mean_ice_2009) |>
  setNames(c("mean_anom", "mean_ice")) |>
  mutate(
    land = is.nan(mean_anom) & mean_ice == 0,
    mean_ice = if_else(land, NA_real_, mean_ice)
  ) |>
  select(land, mean_anom, mean_ice)
```

This follows the task more closely: first compute the two summaries, then combine them, then derive `land`, then replace `mean_ice` with `NA` on land cells.[122]

One more detail

Your `land = is.nan(mean_anom) & mean_ice == 0` is correct according to the sheet, because it says land is where `anom` is missing on all 31 days and `ice` is also missing on all 31 days, which becomes `NaN` for `mean_anom` and `0` for `mean_ice`. Also, resetting `mean_ice` to `NA` on land is exactly what the instructions ask for.[122]

Suggested full 2009 version

This is the version I would recommend:

```
mean_anom_2009 <- st_apply(
  dec_2009["anom"],
  c("x", "y"),
  mean,
  na.rm = TRUE
)

mean_ice_2009 <- st_apply(
  dec_2009["ice"],
  c("x", "y"),
  function(x) mean(replace(x, is.na(x), 0))
)

summary_2009 <- c(mean_anom_2009, mean_ice_2009) |>
  setNames(c("mean_anom", "mean_ice")) |>
  mutate(
    land = is.nan(mean_anom) & mean_ice == 0,
    mean_ice = if_else(land, NA_real_, mean_ice)
  ) |>
  select(land, mean_anom, mean_ice)
```

I changed only two things: I used `replace(x, is.na(x), 0)` as a slightly cleaner way to fill missing ice with zero, and I combined the stars objects first and named them afterward. [123] [122]

For 2010

You should do the exact same sequence with `dec_2010`, just changing the object names. The logic is identical because the task says the same monthly summaries are needed for both December 2009 and December 2010. [122]

One note: if your dimension names are not literally `x` and `y`, use whatever `st_dimensions(dec_2009)` shows, because `st_apply()` matches dimension names exactly. Paste the output of `print(summary_2009)` or `names(summary_2009)`, and I can verify whether the final object structure matches the lab exactly. [123]

⁂

This is the code now:

This is my code,

```
mean_anom_2009 <- st_apply(dec_2009["anom"], c("x", "y"), mean, na.rm = TRUE)
mean_ice_2009 <- st_apply(dec_2009["ice"], c("x", "y"), function(x) {
 mean(replace_na(x, 0))
})

mean_anom_2009
mean_ice_2009

mean_anom_2010 <- st_apply(dec_2010["anom"], c("x", "y"), mean, na.rm = TRUE)
mean_ice_2010 <- st_apply(dec_2010["ice"], c("x", "y"), function(x) {
 mean(replace_na(x, 0))
})

mean_anom_2010
mean_ice_2010

summary_2009 <- c(mean_anom_2009, mean_ice_2009)
summary_2010 <- c(mean_anom_2010, mean_ice_2010)

summary_2009 <- summary_2009 |>
 mutate(
  land = is.nan(mean_anom) & mean_ice == 0,
  mean_ice = if_else(land, NA_real_, mean_ice)
 )

summary_2010 <- summary_2010 |>
 mutate(
  land = is.nan(mean_anom) & mean_ice == 0,
  mean_ice = if_else(land, NA_real_, mean_ice)
 )
```

You're **very close**. The `st_apply()` parts look right, but the part most likely to break is the combined object not having the attribute names `mean_anom` and `mean_ice` before you call `mutate()`.[138] [139]

What is correct

Using `st_apply(dec_2009["anom"], c("x", "y"), mean, na.rm = TRUE)` is the right pattern because the task says to keep the spatial dimensions and collapse `time` into one monthly value per cell. Your `ice` summary is also using the correct idea, since the sheet says missing daily `ice` values should be treated as 0 before averaging.[139] [140] [138]

What to fix

After `summary_2009 <- c(mean_anom_2009, mean_ice_2009)`, the two attributes may not automatically be named `mean_anom` and `mean_ice`. If they are not, then this line can fail:

```
land = is.nan(mean_anom) & mean_ice == 0
```

because `mutate()` needs those names to exist first.

Better version

Use this:

```
mean_anom_2009 <- st_apply(dec_2009["anom"], c("x", "y"), mean, na.rm = TRUE)
mean_ice_2009  <- st_apply(dec_2009["ice"],  c("x", "y"), \(x) mean(replace_na(x, 0))

mean_anom_2010 <- st_apply(dec_2010["anom"], c("x", "y"), mean, na.rm = TRUE)
mean_ice_2010  <- st_apply(dec_2010["ice"],  c("x", "y"), \(x) mean(replace_na(x, 0))

summary_2009 <- c(mean_anom_2009, mean_ice_2009)
names(summary_2009) <- c("mean_anom", "mean_ice")

summary_2010 <- c(mean_anom_2010, mean_ice_2010)
names(summary_2010) <- c("mean_anom", "mean_ice")

summary_2009 <- summary_2009 |>
  mutate(
    land = is.nan(mean_anom) & mean_ice == 0,
    mean_ice = if_else(land, NA_real_, mean_ice)
  ) |>
  select(land, mean_anom, mean_ice)

summary_2010 <- summary_2010 |>
  mutate(
    land = is.nan(mean_anom) & mean_ice == 0,
    mean_ice = if_else(land, NA_real_, mean_ice)
  ) |>
  select(land, mean_anom, mean_ice)
```

Why this matches the task

The sheet says to compute `mean_anom` and `mean_ice` separately with `st_apply()`, combine them with `c()`, then create `land` where `mean_anom` is `NaN` and `mean_ice` is 0, and finally reset `mean_ice` to `NA` on land cells. It also says the final monthly object should contain exactly `land`, `mean_anom`, and `mean_ice`, with no `time` dimension.[138]

One quick check

Run these after your code:

```
names(summary_2009)
names(summary_2010)
st_dimensions(summary_2009)
st_dimensions(summary_2010)
```

You want the names to be exactly:

```
[^11_1] "land" "mean_anom" "mean_ice"
```

and only the spatial dimensions should remain.[139] [138]

The main thing to fix is just the naming step after `c(...)`.

⁂

I am trying to compile and preview the Quarto document but it is taking so long especially on the calculating monthly cell summaries task. Why could this be happening? Is there something wrong with the code?

Most likely, nothing is "wrong" in the sense of incorrect logic — the document is slow because Quarto is probably re-running all the heavy data-import and raster-summary code every time it renders, and `st_apply()` is doing cell-by-cell work across the full spatial grid for 31 daily layers per month.[141] [142] [143]

Main causes

Your monthly-summary chunk is computationally expensive because `st_apply()` keeps the spatial dimensions and applies a function over the remaining one, so for every grid cell it has to process the full time series of daily values. Since you are doing that separately for `anom` and `ice`, and again for both 2009 and 2010, the render is repeating four fairly large reductions over raster cubes.[142] [143] [141]

A second common cause is that Quarto re-executes earlier chunks too, so it is not just the summary step — it may also be re-reading all 62 NetCDF files and rebuilding both `stars` cubes each preview. `read_stars()` is designed to read and combine multiple files into one object, which is convenient but can be slow when repeated during rendering. [144] [145]

One thing in your code

These lines can slow rendering because they print large `stars` objects into the document:

```
mean_anom_2009
mean_ice_2009
mean_anom_2010
mean_ice_2010
```

Printing big raster-like objects during compilation adds overhead on top of the actual calculations. If you only included them for checking, comment them out or replace them with lighter checks such as `names(...)`, `dim(...)`, or `st_dimensions(...)`. [142]

Best fixes

Add caching to the expensive chunks so Quarto does not recompute them every preview unless the code changes. In Quarto chunk options, use:

```
#| cache: true
```

on the import chunk and the monthly-summary chunk.

Also, do not print full objects unless the assignment specifically asks for it. A better pattern is:

```
names(summary_2009)
st_dimensions(summary_2009)
```

instead of printing entire intermediate objects.

Faster version

You can also slightly simplify the `ice` function:

```
mean_ice_2009 <- st_apply(dec_2009["ice"], c("x", "y"), \(x) mean(replace(x, is.na(x
mean_ice_2010 <- st_apply(dec_2010["ice"], c("x", "y"), \(x) mean(replace(x, is.na(x
```

That will not transform performance dramatically, but it avoids a bit of extra overhead from tidyverse-style missing-value helpers inside a function being called many times.

Practical check

Try timing just that section in the console first:

```
system.time({
  mean_anom_2009 <- st_apply(dec_2009["anom"], c("x", "y"), mean, na.rm = TRUE)
  mean_ice_2009  <- st_apply(dec_2009["ice"], c("x", "y"), \(x) mean(replace(x, is.na
})
```

If that alone is slow, the bottleneck is the raster computation itself. If console execution is okay but Quarto preview is much slower, then the main issue is repeated rendering, object printing, and lack of caching.

Use this checklist:

- Add `#| cache: true` to heavy chunks.
- Remove lines that print full `stars` objects.
- Keep imports in their own cached chunk.
- Keep summaries in their own cached chunk.
- Preview only after saving, not on every tiny edit.

Your code logic looks broadly fine; the slowness is much more likely due to repeated raster computation during rendering than a fundamental coding mistake.

⁂

The final map distinguishes land, sea ice, and open ocean. Add a surface attribute to both summary_2009 and summary_2010 by applying the following rules in the stated order:

A cell is "land" when neither an anomaly nor a sea-ice value is available on any day of the month.
A non-land cell is "sea ice" when its mean sea-ice concentration across all 31 days is at least 0.15.
A non-land cell with no available monthly anomaly is also classified as "sea ice". This fallback prevents a missing-anomaly hole at the boundary between the sea-ice and temperature-anomaly masks.
Every remaining cell is "open ocean".

Is this correct?

```
summary_2009 <- summary_2009 |>
  mutate(
    surface = case_when(
      land ~ "land",
```

```
    mean_ice >= 0.15 ~ "sea ice",
    is.na(mean_anom) ~ "sea ice",
    TRUE ~ "open ocean"
   )
  )

summary_2010 <- summary_2010 |>
  mutate(
   surface = case_when(
    land ~ "land",
    mean_ice >= 0.15 ~ "sea ice",
    is.na(mean_anom) ~ "sea ice",
    TRUE ~ "open ocean"
   )
  )

table(summary_2009$surface, useNA = "ifany")
```

Yes — that logic looks correct, and the order in your `case_when()` matches the assignment rules. [146]

Why it is correct

You check `land` first, so cells already identified as land stay land. Then you classify non-land cells with `mean_ice >= 0.15` as `"sea ice"`, which matches the threshold rule. After that, `is.na(mean_anom) ~ "sea ice"` correctly handles the fallback case for non-land cells with no monthly anomaly. Everything else becoming `"open ocean"` matches the final rule. [146]

Small detail

Using `is.na(mean_anom)` here is fine, because in R it also catches `NaN`, which is relevant since your land logic earlier was based on `mean_anom` being `NaN`. That means your fallback line will still pick up missing-anomaly cells at the mask boundary.

Suggested version

I would keep it almost exactly as you wrote it:

```
summary_2009 <- summary_2009 |>
  mutate(
    surface = case_when(
      land ~ "land",
      mean_ice >= 0.15 ~ "sea ice",
      is.na(mean_anom) ~ "sea ice",
      TRUE ~ "open ocean"
    )
  )

summary_2010 <- summary_2010 |>
  mutate(
```

```
    surface = case_when(
      land ~ "land",
      mean_ice >= 0.15 ~ "sea ice",
      is.na(mean_anom) ~ "sea ice",
      TRUE ~ "open ocean"
    )
  )
```

One possible tweak

If `table(summary_2009$surface, useNA = "ifany")` behaves oddly, it may be because `summary_2009$surface` is stored as an array inside the stars object rather than a plain vector. In that case, use:

```
table(as.vector(summary_2009$surface), useNA = "ifany")
```

That is not a logic fix, just a safer way to inspect counts.

One check

You may also want to confirm the final attribute order if the lab expects a specific structure:

```
names(summary_2009)
names(summary_2010)
```

A clean final object would typically include `land`, `mean_anom`, `mean_ice`, and `surface`.

⁂

but why is land not looking at the anom or sea ice value and instead just land ~ "land"

Because `land` is already a **precomputed flag**. In your earlier step, you defined `land` from the monthly summaries, and the lab sheet says a cell is land exactly when `mean_anom` is `NaN` and `mean_ice` is `0`, which recovers the case where both `anom` and `ice` were missing on all 31 days.[147]

So in `case_when()`, this line:

```
land ~ "land"
```

does not mean “ignore anomaly and ice.” It means “if the cell has already been identified as land by the earlier rule, label it as `"land"`.”[147]

Why the check happens earlier

The lab instructions split the work into two stages: first compute `land`, `mean_anom`, and `mean_ice`, then use those summaries to build the final surface classes. That is why `surface` does not need to recalculate from all the original daily `anom` and `ice` values again.[147]

Equivalent expanded version

What you wrote is essentially shorthand for this:

```
surface = case_when(
  is.nan(mean_anom) & mean_ice == 0 ~ "land",
  mean_ice >= 0.15 ~ "sea ice",
  is.na(mean_anom) ~ "sea ice",
  TRUE ~ "open ocean"
)
```

That version and `land ~ "land"` mean the same thing, as long as `land` was correctly created earlier.[147]

Why this is better

Using `land ~ "land"` keeps the code cleaner and avoids repeating the same rule in multiple places. It also makes the order of classification easier to read: first land, then sea ice, then open ocean.[147]

A useful way to think about it is: `land` is not the final label itself; it is a Boolean helper variable that stores the result of the earlier anomaly-and-ice test.

⁂

To plot both Decembers as panels of one faceted map, ggplot2 requires the data to be in a single object. Combine summary_2009 and summary_2010 into one stars object by stacking them along a new year dimension. The resulting object, named dec_summaries, should have the following properties:

Its dimensions are x, y, and year.
Its only attributes are mean_anom and surface. Remove land and mean_ice, which are no longer required.
It contains one 1440 × 720 grid at each of the two positions along year.

Is this correct?

```
#| label: combine-summaries

dec_summaries <- c(
  summary_2009,
  summary_2010,
  along = "year"
)
```

Not quite. Your current code still keeps `land` and `mean_ice`, so it does not yet match the requirement that `dec_summaries` contain only `mean_anom` and `surface`. Also, in the `stars` documentation, `along` is used to arrange arrays along dimension values, so you want to be explicit that the new dimension is `year`. [158]

Better version

Use `select()` first, then combine:

```
dec_summaries <- c(
  summary_2009 |> select(mean_anom, surface),
  summary_2010 |> select(mean_anom, surface),
  along = list(year = c("2009", "2010"))
)
```

This does two important things:

- Removes `land` and `mean_ice`.
- Stacks the two monthly summaries along a new `year` dimension rather than just carrying forward all attributes. [158]

Why your version is incomplete

This version:

```
dec_summaries <- c(
  summary_2009,
  summary_2010,
  along = "year"
)
```

still combines the full objects, so `land` and `mean_ice` remain present. It also may not give you the year values as cleanly as the named `along = list(year = c("2009", "2010"))` form.

Check it after running

Use these checks:

```
names(dec_summaries)
st_dimensions(dec_summaries)
```

You want:

- attributes: `mean_anom`, `surface`
- dimensions: `x`, `y`, `year`

If your lab expects numeric year labels

You can also use:

```
along = list(year = c(2009, 2010))
```

That is often nicer if the `year` dimension is meant to behave like a numeric label.

⁂

First, add an attribute named open_ocean_anom to dec_summaries. Set it equal to mean_anom at cells classified as "open ocean" and to NA elsewhere. Keep mean_anom

unchanged. Restricting the mapped attribute to open-ocean cells ensures that anomalies hidden beneath the sea-ice layer do not influence the scale.
Inspect the distribution of abs(open_ocean_anom) across both years. Set limit to the 99th percentile of its nonmissing values (the open-ocean cells of both panels combined), rounded upward to the next whole degree Celsius. Use this value for both panels.

This is my code,

```
#| label: diverging-fill-scale

dec_summaries <- dec_summaries |>
  mutate(
    open_ocean_anom = if_else(surface == "open ocean", mean_anom, NA_real_)
  )
```

Yes — your `mutate()` line is the correct **first step.** It creates a new attribute that keeps `mean_anom` only where `surface == "open ocean"` and sets every other cell to `NA`, which is exactly what the task is asking for.

Add the attribute

Your code here is fine:

```
dec_summaries <- dec_summaries |>
  mutate(
    open_ocean_anom = if_else(surface == "open ocean", mean_anom, NA_real_)
  )
```

Compute `limit`

Next, you need to:

- take the absolute values,
- combine both years automatically by using the whole `dec_summaries` object,
- ignore missing values,
- get the 99th percentile,
- round upward with `ceiling()`.

A clean way is:

```
vals <- as.numeric(dec_summaries$open_ocean_anom)

limit <- ceiling(
  quantile(abs(vals), probs = 0.99, na.rm = TRUE)
)

limit
```

Why this works

`dec_summaries$open_ocean_anom` pulls that attribute from the full 3D object, so it already includes both years together. `as.numeric()` turns the array values into a plain numeric vector, which makes `abs()`, `quantile()`, and `ceiling()` straightforward.

Full chunk

You can write the whole chunk like this:

```
#| label: diverging-fill-scale

dec_summaries <- dec_summaries |>
  mutate(
    open_ocean_anom = if_else(surface == "open ocean", mean_anom, NA_real_)
  )

vals <- as.numeric(dec_summaries$open_ocean_anom)

limit <- ceiling(
```

```
  quantile(abs(vals), probs = 0.99, na.rm = TRUE)
)

limit
```

One small check

If you want to confirm it looks sensible before plotting, you can also run:

```
summary(vals)
```

or

```
sum(!is.na(vals))
```

That helps verify you are actually using only the open-ocean cells.

Revise the map so that its fill scale has the following properties:

It uses the ColorBrewer RdYlBu diverging palette, reversed so that cold (blue) colors mark negative anomalies and warm (red) colors mark positive ones.
It has symmetric endpoints -limit and limit.
It squishes values outside those limits to the nearest endpoint
instead of treating them as missing—a cell whose anomaly exceeds limit, for instance, is drawn in the deep red of the limit endpoint rather than left blank.
Leave land and sea ice at the default fill for now; they will receive their own layer and colors in the next section.

This was the basic draft,

ggplot() +
 geom_stars(data = dec_summaries, aes(fill = mean_anom)) +
 facet_wrap(vars(year))

Yes — the draft is the right starting point, but you should map `open_ocean_anom`, not `mean_anom`, because the task says only open-ocean anomalies should control the diverging fill scale. To get the required color behavior, use a continuous ColorBrewer diverging scale with palette `"RdYlBu"`, reverse the palette direction, set symmetric limits `c(-limit, limit)`, and use `oob = scales::squish` so values outside the range are clamped to the endpoints instead of dropped. [159] [160] [161]

Revised map

```
ggplot() +
  geom_stars(data = dec_summaries, aes(fill = open_ocean_anom)) +
  facet_wrap(vars(year)) +
  scale_fill_distiller(
    palette = "RdYlBu",
    direction = 1,
    limits = c(-limit, limit),
    oob = scales::squish,
    na.value = NA
  )
```

Why these settings

`scale_fill_distiller()` is the ggplot2 scale intended for continuous ColorBrewer palettes, and `"RdYlBu"` is one of the available diverging palettes. The documentation notes that the distiller scales have `direction = -1` by default, so setting `direction = 1` reverses that default ordering; with `"RdYlBu"`, this gives blue for negative anomalies and red for positive anomalies as required.[162] [159]

Why use `open_ocean_anom`

If you used `mean_anom`, then anomalies in sea-ice cells could still affect the scale or appear in the layer, which is exactly what the task says to avoid. Using `open_ocean_anom` restricts the anomaly layer to open ocean only, leaving land and sea ice blank for now so they can be drawn with separate layers later.[161]

About blank cells

The task says land and sea ice should stay at the default fill for now, so leaving `na.value = NA` is reasonable because non-open-ocean cells in `open_ocean_anom` are `NA` by construction. That means they will not be colored by this anomaly layer, which is what you want before adding the land and sea-ice layer.

One subtle but important fix is the palette direction: for `scale_fill_distiller()`, `direction = 1` is the reversed form relative to its default, which is the version that makes the RdYlBu palette line up with cold blue negatives and warm red positives.[162]

⁂

```
# <q>direction = 1,</q> Shouldn't direction be -1? as it wants reverse
```

Not necessarily — in `scale_fill_distiller()`, the `direction` argument is a bit counterintuitive because the default is already `-1`, so "reverse the palette" does **not** automatically mean you should type `-1`. What matters is the final mapping: negative values should be blue and positive values should be red, and with `palette = "RdYlBu"` that is the outcome you want to check for.[175] [176]

Why `1` can be right

The confusion comes from two different ideas of “reverse”: reversing the named ColorBrewer palette itself, versus reversing how `scale_fill_distiller()` applies it by default. Because the distiller scale already defaults to `direction = -1`, setting `direction = 1` flips that default ordering. [176] [175]

What to use

For your task, use the setting that makes the legend look like this: low/negative anomalies on the left in blue, high/positive anomalies on the right in red. If `direction = -1` gives red for negative values and blue for positive values, then it is the wrong choice for this map even though the word “reverse” appears in the instructions. [175] [176]

Practical rule

So the safest rule is: do not choose `1` or `-1` based on the word “reverse” alone; choose the one that produces blue for negative anomalies and red for positive anomalies in the legend. In the context of `scale_fill_distiller(palette = "RdYlBu")`, that is why `direction = 1` is often the correct setting. [176] [175]

⁂

This is the next section of the labsheet which I need help to understand and how do I approach it?

The map is still drawn directly in longitude and latitude. This equirectangular view stretches high-latitude regions far wider than equatorial regions of equal true area. Because the map invites comparisons of the spatial extent of temperature anomalies and sea ice, redraw it with an equal-area projection. Compare two alternatives:
ProjectionCRS definitionEqual Earth+proj=eqearth +lon_0=180 +ellps=WGS84 +R_A +units=m +no_defs +type=crsMollweide+proj=moll +lon_0=180 +ellps=WGS84 +R_A +units=m +no_defs +type=crs
Both projections use 180° as the central meridian, which places the Pacific Ocean in the center of each panel and aligns the map’s left and right edges with the raster’s existing 0°/360° longitude boundary.
Reprojecting a raster means resampling it: st_warp() lays the cube onto a grid defined in the target CRS, and coord_equal() then draws that already-projected grid. With your chosen projection stored as crs (one of the two definitions above, read with st_crs()), trim the poles, build the target grid explicitly, and resample:

Trim the poles (outside Mollweide's domain) and build the target grid# explicitly, or st_warp() would cover only half of this global source.keep <- which(abs(st_get_dimension_values(dec_summaries, "y")) <= 89.5)trimmed <- dec_summaries[, , keep,]target <- st_as_stars(st_bbox(st_transform(trimmed, crs)), dx = 50000, # 50-km cells (CRS units are meters) dy = 50000)warped <- st_warp(trimmed, target, method = "near")

Nearest-neighbor resampling (method = "near") keeps surface
a factor and does not blur the anomaly across the coastline. Equal
Earth adds one artifact: Its inverse projection wraps data into the
rectangle's corners, which lie outside the round map. Crop the warped
grid to the projected world boundary to blank them. Build that boundary
as a longitude–latitude rectangle spanning the trimmed latitudes, with
its left and right edges a hair inside 0° and 360°. Densify its edges
with st_segmentize(), giving the maximum segment length in
degrees, so that the transformed rectangle follows the projection's
curve, and then transform it to the target CRS. The same boundary traces
the frame that Section 3.12 asks for.
Because the two projections differ only in their CRS, wrap the steps
above in one function of the CRS and call it once per projection. Draw
each map with the two fill layers and scales from Section 3.10, stack the two Decembers in one
column, and replace the default coordinate system with coord_equal().

This section is asking you to do a **raster reprojection workflow**: take your monthly raster summaries, transform them from longitude/latitude into an equal-area projected coordinate system, resample them onto a new projected grid, and then plot the projected raster with `coord_equal()`. The point is to make area comparisons more honest, because the current lat-long view stretches high latitudes and makes polar features look wider than they really are. [177] [178] [179] [180]

What it means

Your current `dec_summaries` raster is on a geographic grid with coordinates in degrees, so equal-sized raster cells on the map are not equal area on Earth. The lab asks you to compare two equal-area projections, Equal Earth and Mollweide, both centered on longitude 180 degrees so the Pacific stays in the middle and the raster's 0 to 360 longitude layout stays aligned with the panel edges. [179] [180] [181]

Reprojecting a raster is different from reprojecting points or polygons because the cell values have to be **resampled** onto a new grid in the target CRS. The `stars` function `st_warp()` does exactly that by warping a source raster onto a target raster grid that you define first. [178] [177]

Why the steps exist

The lab tells you to trim the poles first because some global equal-area projections, especially Mollweide and the Equal Earth workflow described here, can behave badly at the extreme poles or produce invalid edge artifacts. It then tells you to build the target grid explicitly with `st_as_stars(st_bbox(...), dx = 50000, dy = 50000)` because otherwise `st_warp()` may choose an extent or alignment that is not what you want for a full global raster. [181] [178]

Using `method = "near"` means nearest-neighbor resampling, which is important here because one of your attributes, `surface`, is categorical. Nearest-neighbor preserves category labels and also avoids smoothing anomaly values across coastlines in the way bilinear interpolation could. [178] [181]

How to approach it

Make a function that takes one CRS as input and returns a warped version of `dec_summaries`. The function should:

- define the CRS with `st_crs()`,
- remove the polar rows beyond 89.5 degrees,
- transform the trimmed object's bounding box into the target CRS,
- build a 50 km target raster grid,
- warp the trimmed raster onto that grid with nearest-neighbor resampling. [177] [181] [178]

A direct translation of the lab logic looks like this:

```
warp_monthly <- function(crs) {
  keep <- which(abs(st_get_dimension_values(dec_summaries, "y")) <= 89.5)
  trimmed <- dec_summaries[, , keep, ]

  target <- st_as_stars(
    st_bbox(st_transform(trimmed, crs)),
    dx = 50000,
    dy = 50000
  )

  warped <- st_warp(trimmed, target, method = "near")
  warped
}
```

Why Equal Earth needs cropping

The lab warns that Equal Earth can wrap data into the corners of the rectangular projected extent even though those corners are outside the rounded outline of the world in that projection. So after warping, you need a projected world boundary polygon and then crop or mask the raster against it so those corner artifacts become blank. [181]

The boundary starts as a lon-lat rectangle that matches your trimmed world extent, but the left and right edges should be slightly inside 0 and 360 degrees to avoid seam problems at the

antimeridian-like boundary in your 0 to 360 raster. The edges are then densified with `st_segmentize()` before projection so the transformed boundary follows the curve of the map rather than becoming a coarse straight-edged shape. [181]

Conceptually:

```
make_boundary <- function(crs) {
  lat_rng <- range(st_get_dimension_values(dec_summaries, "y"))
  lat_rng <- lat_rng[abs(lat_rng) <= 89.5]

  bb_ll <- st_bbox(c(
    xmin = 0.001,
    xmax = 359.999,
    ymin = min(lat_rng),
    ymax = max(lat_rng)
  ), crs = st_crs(4326))

  boundary <- st_as_sfc(bb_ll) |>
    st_segmentize(dfMaxLength = 1) |>
    st_transform(crs)

  boundary
}
```

Then you would use that boundary to crop or mask the warped raster so the invalid corners disappear. [181]

What the final structure is

You need to run the function twice, once for Equal Earth and once for Mollweide, because the only thing changing is the CRS. After that, you make the same two-layer faceted map for each projected result, keep the two Decembers stacked in one column, and replace the default coordinate system with `coord_equal()` because the data are already projected into planar coordinates. [180] [181]

The two CRS definitions from your lab are:

```
crs_eqearth <- st_crs("+proj=eqearth +lon_0=180 +ellps=WGS84 +R_A +units=m +no_defs +
crs_moll    <- st_crs("+proj=moll +lon_0=180 +ellps=WGS84 +R_A +units=m +no_defs +typ
```

Practical mindset

So the workflow is:

1. Start with `dec_summaries`.
2. Trim polar rows.
3. Build a projected target grid.
4. Warp with nearest neighbor.

5. Build projected world boundary.
6. Crop or mask corner artifacts.
7. Plot with your existing anomaly and surface layers using `coord_equal()`.[177] [178] [181]

The key idea is that **projection for rasters is not just changing CRS labels.** You must create a new raster grid in the new CRS and resample onto it.[178]

Would you like me to write the full R function for both projections in the exact package style your lab allows: `tidyverse`, `stars`, `scales`, `ggnewscale`, and `units`?

⁂

<q>bb_ll <- st_bbox(c(

```
xmin = 0.001,
xmax = 359.999,
ymin = min(lat_rng),
ymax = max(lat_rng)
```

), crs = st_crs(4326))</q> How does this work?

Yes — that line is just **constructing a rectangular bounding box in longitude/latitude coordinates.** It does not create map data by itself; it creates an object that says "my rectangle runs from this minimum longitude to this maximum longitude, and from this minimum latitude to this maximum latitude."[193] [194]

Line by line

`st_bbox()` can be used as a constructor: if you give it a numeric vector with named elements `xmin`, `ymin`, `xmax`, and `ymax`, it returns a `bbox` object with those four limits plus a CRS attribute. So this code is defining the left, right, bottom, and top edges of a rectangle.[193] [195]

```
bb_ll <- st_bbox(c(
  xmin = 0.001,
  xmax = 359.999,
  ymin = min(lat_rng),
  ymax = max(lat_rng)
), crs = st_crs(4326))
```

What the numbers mean

`xmin = 0.001` and `xmax = 359.999` are the longitude limits of the rectangle, and `ymin` and `ymax` are the latitude limits. `st_crs(4326)` assigns the rectangle the WGS84 geographic CRS, which is the standard longitude/latitude system known as EPSG:4326.[193] [194]

In your lab context, `min(lat_rng)` and `max(lat_rng)` mean "use the lowest and highest trimmed latitudes that remain after removing the polar rows." The small offset from 0 and 360 is there so the boundary sits just inside the raster seam instead of exactly on it.

Why make a bbox first

A `bbox` is a convenient way to describe an extent before turning it into geometry. The `sf` documentation notes that `st_as_sfc()` has a method for `bbox` objects, which means you can convert that bounding box into an actual polygon rectangle afterward. [193] [196]

So the typical flow is:

```
bb_ll
# bounding box definition only

st_as_sfc(bb_ll)
# rectangle polygon in lon/lat
```

Why this matters here

Once you convert the bounding box to a polygon, you can densify its edges and then project it into Equal Earth or Mollweide so it becomes the curved world boundary used for masking or framing. That is why the code starts with a simple lon-lat rectangle: it is an easy template for building the final projected outline. [193] [196] [197]

A simple way to read the whole line is: “Create a WGS84 lon-lat rectangle spanning almost 0 to 360 degrees in longitude and the trimmed latitude range, and store it as `bb_ll`.” [194] [193]

If you want, I can also explain the very next line, `st_as_sfc(bb_ll) |> st_segmentize(...) |> st_transform(crs)`, piece by piece.

⁂

This is a requirement in the labsheet:

Nearest-neighbor resampling (method = "near") keeps surface a factor and does not blur the anomaly across the coastline. Equal Earth adds one artifact: Its inverse projection wraps data into the rectangle’s corners, which lie outside the round map. Crop the warped grid to the projected world boundary to blank them. Build that boundary as a longitude–latitude rectangle spanning the trimmed latitudes, with its left and right edges a hair inside 0° and 360°. Densify its edges with st_segmentize(), giving the maximum segment length in degrees, so that the transformed rectangle follows the projection’s curve, and then transform it to the target CRS. The same boundary traces the frame that Section 3.12 asks for.

How do i use st_segmentize and implement it in this case?

Yes — in this task, `st_segmentize()` is used to **densify** the boundary before projection, not to crop by itself. It adds extra vertices along each edge of your lon-lat rectangle, which matters because when that rectangle is transformed to Equal Earth or Mollweide, the many added vertices let the projected edge bend smoothly instead of being defined only by its four corners. [208] [209] [210]

What `st_segmentize()` does

`st_segmentize(x, dfMaxLength = ...)` makes sure no line segment in the geometry is longer than the specified maximum length, so it inserts intermediate points where needed. It does **not** split the object into many separate features; it keeps the same geometry type, just with more vertices.[211] [212] [208]

In your case, the input is a world rectangle in longitude-latitude space, so the lab wants the maximum segment length given in degrees before projection. That matches the instruction to densify the lon-lat rectangle first and only then transform it to the target CRS.[210]

Why you need it here

If you transform a rectangle made from only four corners, the projected boundary can be too coarse because the edges are represented by very few points. Densifying the edges first gives the transformation enough intermediate points to trace the curved outline of the world boundary correctly in Equal Earth or Mollweide.[209] [213] [210]

That is especially important for your crop mask, because the lab says the same transformed boundary should be used to blank the Equal Earth corner artifacts and later serve as the map frame.[210]

How to implement it

The workflow is:

1. Build a lon-lat bounding box just inside 0 and 360 degrees.[210]
2. Convert that bounding box to an `sfc` polygon.[214]
3. Densify it with `st_segmentize(dfMaxLength = 1)` or another small degree value.[211] [210]
4. Transform that densified polygon into your chosen projection.[213] [210]

Here is the pattern:

```
make_boundary <- function(crs) {
  yvals <- st_get_dimension_values(dec_summaries, "y")
  lat_rng <- range(yvals[abs(yvals) <= 89.5])

  bb_ll <- st_bbox(
    c(
      xmin = 0.001,
      xmax = 359.999,
      ymin = lat_rng[^21_1],
      ymax = lat_rng[^21_2]
    ),
    crs = st_crs(4326)
  )

  boundary <- bb_ll |>
    st_as_sfc() |>
    st_segmentize(dfMaxLength = 1) |>
    st_transform(crs)
```

```
  boundary
}
```

Here, `dfMaxLength = 1` means “add enough vertices so no edge segment is longer than 1 degree” while the geometry is still in lon-lat coordinates. That is exactly the kind of densification your lab sheet is asking for. [211] [210]

How to think about `dfMaxLength`

A smaller value means more vertices and a smoother projected curve, while a larger value means fewer vertices and a rougher curve. Since your lab explicitly says to give the maximum segment length in degrees, values like `1` or `0.5` are sensible starting points for a global boundary. [215] [210]

How it fits into the full reprojection

After you build the boundary, you warp the raster first and then crop or mask the warped raster with this projected boundary so the corners outside the round map become blank. The lab specifically says to use nearest-neighbor resampling for the warp and then crop the warped grid to the projected world boundary. [216] [210]

So conceptually, the sequence is:

```
crs_eqearth <- st_crs("+proj=eqearth +lon_0=180 +ellps=WGS84 +R_A +units=m +no_defs +

keep <- which(abs(st_get_dimension_values(dec_summaries, "y")) <= 89.5)
trimmed <- dec_summaries[, , keep, ]

target <- st_as_stars(
  st_bbox(st_transform(trimmed, crs_eqearth)),
  dx = 50000,
  dy = 50000
)

warped <- st_warp(trimmed, target, method = "near")
boundary <- make_boundary(crs_eqearth)
```

The main role of `st_segmentize()` in all of this is just to make `boundary` curved correctly after projection. [209] [210]

One simple intuition

Think of the original rectangle as a cardboard frame with only four corner pins. `st_segmentize()` adds many pins along the edges, so when you bend the frame into a new projection, the edges follow the curve smoothly instead of cutting across it. [208] [209]

If you want, I can write the full reusable function that does **trim -> target grid -> warp -> boundary -> crop -> plot** for both Equal Earth and Mollweide.

⁂

what ddoes coord_equal() do?

`coord_equal()` makes the x and y axes use the same physical scale, so one unit in the x direction is drawn the same length as one unit in the y direction. In other words, it fixes the aspect ratio to 1:1 instead of letting the plot stretch horizontally or vertically to fill space. [226] [227] [228]

Why that matters

Without `coord_equal()`, ggplot can stretch the panel, which can distort shapes and distances on a projected map. With `coord_equal()`, squares stay square and circles stay circular in projected coordinates, which is especially important after you have already reprojected your raster into meters. [227] [228] [226]

In your lab

After `st_warp()`, your raster is no longer in longitude and latitude; it is on a projected x-y grid in planar units. The lab therefore asks you to use `coord_equal()` so ggplot draws that projected grid faithfully instead of stretching it. [229] [226]

Simple intuition

Think of it as telling ggplot, “Do not resize the map differently in width and height; keep the geometry honest.” In fact, `coord_equal()` is essentially a shortcut for a fixed coordinate ratio of 1. [228] [226]

A tiny example is:

```
ggplot() +
  geom_stars(data = warped, aes(fill = open_ocean_anom)) +
  coord_equal()
```

That does not change the data values or projection; it only changes how the already projected coordinates are drawn on the page. [226] [227]

⁂

Mollweide projection vs Equal earth projection in terms of map projections

Both Mollweide and Equal Earth are **equal-area** pseudocylindrical world map projections, so they preserve relative area and are appropriate when you want fair comparisons of spatial extent on global thematic maps. The main difference is that Mollweide is an older oval projection with stronger shape distortion near the outer edges, while Equal Earth is a newer design intended to preserve area while looking more visually natural. [237] [238] [239] [240] [241]

Core difference

The Mollweide projection represents the world as a 2:1 ellipse, with the equator as a straight horizontal line and the central meridian as a straight vertical line. Equal Earth is also pseudocylindrical and equal-area, but it was introduced in 2018 and was designed to give a more balanced, Robinson-like appearance while still keeping area correct. [238] [242] [240] [243] [241]

Distortion pattern

Mollweide preserves area well, but shapes, directions, angles, and distances are generally distorted, with especially noticeable distortion toward the outer edges of the ellipse and at high latitudes. Equal Earth also distorts shapes, because no equal-area world projection can preserve everything, but it is often chosen because its overall look is considered more natural for world-scale thematic mapping. [242] [240] [243] [241] [237] [238]

When to use each

Mollweide is a classic choice for scientific and thematic world maps, especially when you want an unmistakable oval world and accurate area comparisons. Equal Earth is often a strong choice for modern thematic maps when you still need equal area but want the map to be easier on the eye and less visually harsh at the edges. [239] [240] [241] [237] [238]

For your lab

For climate anomaly and sea-ice maps, both projections are defensible because the assignment is about comparing spatial extent, and both preserve area. If you want a more traditional scientific look, choose Mollweide; if you want a more modern and visually balanced presentation, choose Equal Earth. In your case, the lab note about Equal Earth corner artifacts is also practical: Equal Earth may need extra masking outside the rounded world boundary after reprojection. [240] [241] [244] [237] [239]

A simple way to remember it is: Mollweide is the classic oval equal-area map, while Equal Earth is the newer equal-area map designed to look more natural. [241] [239]

⁂

The team would like to use the molewiede projection. What are the justifications that can be used, advantages and disadvantages

Yes — Mollweide is a defensible choice for your lab because it is an **equal-area** pseudocylindrical projection intended for small-scale thematic world maps, so it supports fair comparison of the spatial extent of sea ice and temperature anomalies. That makes it a strong fit for climate mapping, where misleading area inflation would weaken the interpretation of global patterns. [253] [254] [255] [256]

Justifications

Your team can justify Mollweide by saying the map's purpose is to compare the areal extent of climate features across the world, and Mollweide preserves relative area across the globe. It is specifically described as appropriate for thematic world maps and for area-based analysis, which matches anomaly and sea-ice mapping well. [257] [254] [256]

A second justification is that Mollweide has a long history of use in scientific and thematic mapping, including global climate-style displays. Its oval full-world layout also presents the entire globe in one continuous frame, which works well for synoptic comparison across ocean basins. [258] [254] [257] [253]

Advantages

The main advantage is accurate area representation, so large high-latitude regions are not exaggerated the way they would be in projections such as Mercator. For a climate map, that makes comparisons of sea-ice coverage or warm and cool anomaly regions more honest. [254] [255] [257] [253]

Another advantage is that Mollweide is widely regarded as suitable for global distribution maps such as climate, population, vegetation, or other thematic surfaces. Its regular, symmetric oval layout can also make global patterns feel orderly and easy to compare between panels. [259] [256] [253] [254]

Disadvantages

The main drawback is that Mollweide does **not** preserve shape, angle, direction, or distance, so coastlines and landmasses become increasingly distorted away from the center and especially near the outer edges of the ellipse. That means it is good for extent comparisons, but not ideal if your audience needs locally realistic shapes. [256] [257] [253] [254]

A second drawback is that distortion becomes severe near the map margins and at high latitudes, even though area remains correct. Compared with Equal Earth, Mollweide is often seen as less visually natural and a bit harsher in outline and edge distortion. [260] [261] [258] [254] [256]

Report wording

A concise justification for your report could be: "We chose the Mollweide projection because it is an equal-area world projection suited to thematic mapping, allowing fair visual comparison of the spatial extent of sea-ice and temperature-anomaly patterns." You can then acknowledge the trade-off by adding: "Its limitation is increased shape and distance distortion toward the outer edges, but area preservation is the more important property for this map's purpose." [257] [254] [256]

Would you like a shorter 2-3 sentence version written in report style for direct pasting into your lab?

⁂

This is what I wrote, what do you think?

d. Which projection makes the contrast between the two Decembers more visually prominent?

For the Equal Earth projection, it fills more of the panel and keeps a shape close to a familiar world map, so the eye can scan and compare the two stacked panels faster, and the red/blue anomaly clusters read as slightly "bolder" against a fuller frame.

For the Mollweide projection, its compact, symmetric oval keeps the tropics and mid-latitudes closer to the vertical center of the panel, arguably making the stacked comparison between the two Decembers easier since there's less peripheral distortion pulling the eye toward the edges.

Therefore, the Mollweide projection makes the contrast more visually prominent.

It's a **good** start, but the argument feels slightly undecided because both paragraphs praise each projection before you conclude in favor of Mollweide. If you want the answer to sound stronger, make the Mollweide paragraph do more of the persuasive work and link the conclusion directly to "visual prominence," not just general readability. [269] [270]

What to improve

Your Equal Earth paragraph currently gives a fairly strong case for Equal Earth, so it weakens the final conclusion. Since Mollweide is a compact, symmetric oval projection, you can emphasize that its shape reduces edge distraction and focuses attention on the anomaly patterns, even though Equal Earth often looks more familiar and visually natural overall. [270] [271] [272] [269]

Tighter version

You could rewrite it like this:

> **The Mollweide projection makes the contrast between the two Decembers more visually prominent.** Although Equal Earth fills more of the panel and resembles a familiar world map, the Mollweide projection's compact, symmetric oval gives stronger visual focus to the central climate patterns. Because it reduces peripheral emphasis at the map edges, the differences in sea-ice extent and temperature-anomaly clusters between the two months stand out more clearly. [269] [270]

Small wording tip

If this is for a lab report, avoid words like "arguably" unless you want to sound cautious. For this kind of question, a direct comparative sentence usually reads better than a balanced discussion followed by a sudden conclusion.

⁂

```{r}
#| label: continuous-scale

keep <- which(abs(st_get_dimension_values(data, "y")) <= 89.5)
trimmed <- dec_summaries[, , keep, ]


target <- st_as_stars(
  st_bbox(st_transform(trimmed, mollweide_crs)),
  dx = 50000, # 50 km cells (CRS units are meters)
  dy = 50000
)

warped <- st_warp(trimmed, target, method = "near")

bbox_latlon <- st_bbox(c(xmin = 0.001, xmax = 359.999, ymin = -89.5, ymax = 89.5), crs = 4326)
world_boundary <- st_as_sfc(bbox_latlon) |>
  st_segmentize(dfMaxLength = 1) |>
  st_transform(mollweide_crs)

warped_cropped <- st_crop(warped, world_boundary)

ggplot() +
  geom_stars(data = warped_cropped, aes(fill = open_ocean_anom)) +
  scale_fill_fermenter(
    palette = "RdYlBu",
    direction = -1,
    limits = c(-limit, limit),
    oob = scales::squish,
    name = "Anomaly (°C)"
  ) +
  ggnewscale::new_scale_fill() +
  geom_stars(data = warped_cropped, aes(fill = surface)) +
  facet_wrap(vars(year), ncol = 1) +
  scale_fill_manual(
    values = c(
      land = "grey70",
      `sea ice` = "white",
      `open ocean` = "transparent"
    ),
    breaks = c("land", "sea ice"),
    labels = c("Land", "Sea-ice concentration >= 15%"),
    name = NULL
  ) +
  coord_equal()
```

Why is this giving me the error,

Error in `UseMethod()`:
! no applicable method for 'expand_dimensions' applied to an object of class "function"
Backtrace:
■

1. ├─base::which(abs(st_get_dimension_values(data, "y")) <= 89.5)
2. └─stars::st_get_dimension_values(data, "y")
3. └─stars::expand_dimensions(.x, ..., max = max, center = center)

The error is happening because you wrote `st_get_dimension_values(data, "y")`, but `data` here is not your stars object — it is being interpreted as some other object, apparently a **function**. The `st_get_dimension_values()` method expects a `stars` object, and the traceback says it received an object of class `"function"` instead. [273] [274]

Exact problem

This line is the issue:

```
keep <- which(abs(st_get_dimension_values(data, "y")) <= 89.5)
```

You probably meant to use `dec_summaries`, not `data`. Since `data` is not your raster cube, `stars` cannot extract dimension values from it. [274] [273]

Fix

Change that line to:

```
keep <- which(abs(st_get_dimension_values(dec_summaries, "y")) <= 89.5)
```

Then the next line will make sense:

```
trimmed <- dec_summaries[, , keep, ]
```

Why the error message looks strange

Internally, `st_get_dimension_values()` tries to expand the dimensions of the object you pass in. Because you passed a function instead of a `stars` object, the internal method dispatch fails and produces the message about no applicable method for `expand_dimensions` on class `"function"`. [275] [276] [273]

One more likely issue

You are using:

```
scale_fill_fermenter(
```

but your anomaly layer is continuous numeric data, not binned data. The ggplot2 Brewer documentation distinguishes `distiller` for continuous colour scales and `fermenter` for binned colour scales, so for `open_ocean_anom` you probably want:

```
scale_fill_distiller(
  palette = "RdYlBu",
  direction = 1,
  limits = c(-limit, limit),
  oob = scales::squish,
  name = "Anomaly (°C)"
)
```

instead of `scale_fill_fermenter()`.[277] [278]

Corrected version

```
keep <- which(abs(st_get_dimension_values(dec_summaries, "y")) <= 89.5)
trimmed <- dec_summaries[, , keep, ]

target <- st_as_stars(
  st_bbox(st_transform(trimmed, mollweide_crs)),
  dx = 50000,
  dy = 50000
)

warped <- st_warp(trimmed, target, method = "near")

bbox_latlon <- st_bbox(
  c(xmin = 0.001, xmax = 359.999, ymin = -89.5, ymax = 89.5),
  crs = 4326
)

world_boundary <- st_as_sfc(bbox_latlon) |>
  st_segmentize(dfMaxLength = 1) |>
  st_transform(mollweide_crs)

warped_cropped <- st_crop(warped, world_boundary)

ggplot() +
  geom_stars(data = warped_cropped, aes(fill = open_ocean_anom)) +
  scale_fill_distiller(
    palette = "RdYlBu",
    direction = 1,
    limits = c(-limit, limit),
    oob = scales::squish,
    name = "Anomaly (°C)"
  ) +
  ggnewscale::new_scale_fill() +
  geom_stars(data = warped_cropped, aes(fill = surface)) +
  facet_wrap(vars(year), ncol = 1) +
  scale_fill_manual(
    values = c(
      land = "grey70",
```

```
      `sea ice` = "white",
      `open ocean` = "transparent"
    ),
    breaks = c("land", "sea ice"),
    labels = c("Land", "Sea-ice concentration >= 15%"),
    name = NULL
  ) +
  coord_equal()
```

One final note

`st_crop()` crops by bounding box, so if your goal is to blank the curved outside corners of the Equal Earth map, cropping alone may not fully mask non-rectangular areas. For Mollweide this may still be okay depending on the output, but for a truly curved mask you may later need intersection or masking rather than only bbox crop. [279]

The immediate error, though, is simply `data` versus `dec_summaries`.

⁂

Currently my plot is to Meet the following requirements:

Stack both December maps in one column, in the equal-area projection you chose, so each map fills the figure's width.
Keep the diverging open-ocean anomaly fill and the land and sea-ice surface overlay from Section 3.10,
with symmetric limits centered on 0 °C; in the surface legend, name the
sea-ice class by its concentration threshold of at least 15%.
Leave blank the corners that fall outside the round map, and trace the projection boundary with a thin frame.
Mark the equator with a faint line; the El Niño and La Niña signal runs along it.
Label each panel strip with the month, year, and phase—for example, "December 2009 (El Niño)".
Give the anomaly legend a title that carries its unit, °C.
Outline the anomaly legend and the surface keys with a thin dark
line, so that the white sea-ice key stays distinct from the white
background.
Add a title and subtitle stating what is mapped and what the anomaly
is measured against, and a caption that credits the data source.
Strip the non-data ink—axis titles, ticks, and grid—so the maps carry the figure.
Optionally, label the major oceans (Pacific, Atlantic, and Indian)
on the map face; keep the text legible over any color, for example with a
semitransparent box behind it (geom_label()).

What was I missing from my previous ggplot?

You had the **core map layers**, but you were still missing several presentation requirements from the lab sheet. Your earlier plot already had the projected raster, faceting, anomaly fill, surface overlay, and `coord_equal()`, but it did not yet fully implement the frame, equator, panel labels, full annotation, and legend styling requirements. [291] [292] [293]

Missing items

From the lab requirements, these were still missing or only partly addressed:

- A thin frame tracing the projected world boundary; you built `world_boundary`, but you did not draw it as a visible layer. [293]
- A faint equator line. [293]
- Facet labels in the exact form “December 2009 (El Niño)” and “December 2010 (La Niña)”; your `facet_wrap(vars(year))` only uses the year. [293]
- A title, subtitle, and caption. [293]
- Removal of non-data ink such as axis text, ticks, titles, and grid; `coord_equal()` does not do that by itself. [294] [293]
- Legend key outlines so the white sea-ice key remains visible on the white background. [291] [293]
- Potentially the anomaly legend outline as well, depending on how your ggplot theme currently renders the colour bar. [292] [293]

One likely issue

You also said the corners outside the round map should be blank, but your earlier use of `st_crop()` only crops by bounding box, not by the curved shape of the projected boundary. So even if the plot looks closer, it may not yet truly mask the outside-corner areas in the way the lab describes. [293]

What the finished plot needs

A more complete plot would add:

- `geom_sf(data = world_boundary, fill = NA, color = ...)` for the frame.
- An equator layer, created as a line in lon-lat and transformed to Mollweide.
- Better facet labels via a labeller or a prepared facet variable.
- `labs(title = ..., subtitle = ..., caption = ...)`.
- `theme_void()` or equivalent theme cleanup.
- Legend styling with bordered keys and a bordered colour bar. [292] [291] [293]

Practical checklist

So compared with your previous ggplot, you were mainly missing:

- the visible world boundary frame,
- the equator line,
- custom facet strip text,
- plot title/subtitle/caption,
- theme cleanup,
- legend border styling,
- and possibly a true curved mask instead of only rectangular crop.[293]

How to think about it

Your previous plot mostly handled the **data drawing** part. The remaining tasks are mostly about **cartographic finishing**: layout, annotation, boundary handling, and legend polish.[293]

If you want, I can now write a near-final ggplot block that includes all the missing pieces for the Mollweide version.

⁂

Currently this is the code I am trying to run for this task, however, I have been running the plot for 10+ minutes and it still hasn't rendered,

```
#| label: data-preparation

keep <- which(abs(st_get_dimension_values(dec_summaries, "y")) <= 89.5)
trimmed <- dec_summaries[, , keep, ]

target <- st_as_stars(
  st_bbox(st_transform(trimmed, mollweide_crs)),
  dx = 50000, # 50 km cells (CRS units are meters)
  dy = 50000
)

warped <- st_warp(trimmed, target, method = "near")

bbox_latlon <- st_bbox(c(xmin = 0.001, xmax = 359.999, ymin = -89.5, ymax = 89.5), cr
world_boundary <- st_as_sfc(bbox_latlon) |>
  st_segmentize(dfMaxLength = 3) |>
  st_transform(mollweide_crs)

warped_cropped <- st_crop(warped, world_boundary)

warped_cropped <- warped_cropped |>
  st_set_dimensions(
```

```
    "year",
    values = case_when(
      as.character(st_get_dimension_values(warped_cropped, "year")) == "2009" ~ "Dece
      as.character(st_get_dimension_values(warped_cropped, "year")) == "2010" ~ "Dece
      TRUE ~ as.character(st_get_dimension_values(warped_cropped, "year"))
    )
  )

equator_line <- st_linestring(matrix(c(seq(0.001, 359.999, length.out = 100), rep(0,
  st_sfc(crs = 4326) |>
  st_segmentize(dfMaxLength = 3) |>
  st_transform(mollweide_crs)

ocean_labels <- st_sf(
    name = c("PACIFIC OCEAN", "ATLANTIC OCEAN", "INDIAN OCEAN"),
    geometry = st_sfc(
      st_point(c(180, 0)),
      st_point(c(330, 0)),
      st_point(c(80, -20)),
      crs = 4326
    )
  ) |>
  st_transform(mollweide_crs)

ocean_coords <- as.data.frame(st_coordinates(ocean_labels))
ocean_coords$name <- ocean_labels$name

equator_df <- as.data.frame(st_coordinates(equator_line))

world_boundary_df <- as.data.frame(st_coordinates(world_boundary))
```

```
#| label: continuous-scale-plot

ggplot() +
  geom_stars(data = warped_cropped, aes(fill = open_ocean_anom)) +
  scale_fill_fermenter(
    palette = "RdYlBu",
    direction = -1,
    limits = c(-limit, limit),
    oob = scales::squish,
    name = "Anomaly (°C)",
    guide = guide_colorsteps(frame.colour = "black", frame.linewidth = 0.3, ticks.col
  ) +
  ggnewscale::new_scale_fill() +
  geom_stars(data = warped_cropped, aes(fill = surface)) +
  facet_wrap(vars(year), ncol = 1) +
  scale_fill_manual(
    values = c(
      land = "grey70",
      `sea ice` = "white",
      `open ocean` = "transparent"
    ),
    breaks = c("land", "sea ice"),
    labels = c("Land", "Sea-ice concentration >= 15%"),
```

```
    name = NULL,
    guide = guide_legend(override.aes = list(colour = "black", linewidth = 0.3))
  ) +
  geom_path(
    data = equator_df, aes(x = X, y = Y),
    colour = "grey60", linetype = "dashed", linewidth = 0.3
  ) +
  geom_polygon(
    data = world_boundary_df, aes(x = X, y = Y, group = L1),
    fill = NA, colour = "black", linewidth = 0.4
  ) +
  geom_label(
    data = ocean_coords,
    aes(x = X, y = Y, label = name),
    fill = alpha("white", 0.6),
    color = "grey20",
    size = 2.8,
    fontface = "bold",
    label.size = 0,
    label.padding = unit(0.15, "lines")
  ) +
  coord_equal() +
  labs(
    title = "Sea Surface Temperature Anomalies During El Nino and La Nina",
    subtitle = "Monthly mean departure from 1971-2000 climatology",
    caption = "Source: NOAA Optimum Interpolation Sea Surface Temperature (OISST) v2
  ) +
  theme_void() +
  theme(
    strip.text = element_text(face = "bold", size = 11, margin = margin(b = 5, t = 5)
    plot.title = element_text(face = "bold", size = 14, hjust = 0),
    plot.subtitle = element_text(size = 10, color = "grey30", hjust = 0, margin = mar
    plot.caption = element_text(size = 8, color = "grey40", hjust = 1, margin = margi
    legend.position = "right",
    plot.margin = margin(10, 10, 10, 10)
  )
```

The long runtime is probably coming from the **data preparation**, especially `st_warp()`, not from the final `ggplot()` lines alone. `st_warp()` resamples the whole raster cube onto a new grid in the target CRS, and plotting reprojected rasters can also be slow when the result is large. [302] [303] [304]

Most likely bottleneck

This part is the heaviest section:

```
target <- st_as_stars(
  st_bbox(st_transform(trimmed, mollweide_crs)),
  dx = 50000,
  dy = 50000
)
```

```
warped <- st_warp(trimmed, target, method = "near")
```

You are warping a global raster with multiple attributes and two year slices onto a 50 km grid, which can create a very large output object. Since Quarto reruns chunks during rendering, this can easily take many minutes. [304] [302]

Likely secondary issues

You are also drawing **two full raster layers** with `geom_stars()`, plus polygon and path overlays, which adds plotting cost after the warp is finished. The `geom_stars()` documentation notes that it dispatches to raster-, tile-, or sf-based drawing depending on the geometry, and large raster objects can still be slow to render. [303] [305]

Another issue is that your anomaly scale uses `scale_fill_fermenter()`, which is a **binned** Brewer scale, while your anomaly field is continuous. The ggplot2 documentation distinguishes `scale_fill_distiller()` for continuous Brewer scales and `scale_fill_fermenter()` for binned scales, so your current choice is not ideal for the lab's continuous diverging anomaly field. [306] [307] [308]

Small code issues

You may also have hidden formatting issues from copied code: I can see non-breaking spaces in some lines such as before `target <- ...`. Those sometimes cause odd behavior in Quarto or R editors, though they usually do not create a 10-minute hang by themselves.

Also, this line is okay logically but repeats `st_get_dimension_values()` three times:

```
values = case_when(
  as.character(st_get_dimension_values(warped_cropped, "year")) == "2009" ~ ...,
  as.character(st_get_dimension_values(warped_cropped, "year")) == "2010" ~ ...,
  TRUE ~ as.character(st_get_dimension_values(warped_cropped, "year"))
)
```

That is not the main performance issue, but it is unnecessary repeated work.

What to do first

Test the chunks **step by step in the console**, not through full Quarto render. In particular, run timing on the warp:

```
system.time({
  keep <- which(abs(st_get_dimension_values(dec_summaries, "y")) <= 89.5)
  trimmed <- dec_summaries[, , keep, ]

  target <- st_as_stars(
    st_bbox(st_transform(trimmed, mollweide_crs)),
    dx = 50000,
    dy = 50000
  )
```

```
  warped <- st_warp(trimmed, target, method = "near")
})
```

If this is the slow part, then the plot is not the true culprit.

Best fixes

1. Cache the heavy chunk in Quarto:

```
#| cache: true
```

2. Try a coarser target grid first, for example 100 km instead of 50 km:

```
dx = 100000,
dy = 100000
```

That should greatly reduce the number of projected cells and speed up both warping and plotting. [302] [304]

3. Switch the anomaly scale to `scale_fill_distiller()` because your anomaly values are continuous. [308] [306]
4. Render the plot without ocean labels and boundary first, then add layers back after the raster renders.

Cleaner year labeling

You can simplify this part:

```
year_vals <- as.character(st_get_dimension_values(warped_cropped, "year"))

year_labels <- case_when(
  year_vals == "2009" ~ "December 2009 (El Niño)",
  year_vals == "2010" ~ "December 2010 (La Niña)",
  TRUE ~ year_vals
)

warped_cropped <- warped_cropped |>
  st_set_dimensions("year", values = year_labels)
```

That will not transform runtime dramatically, but it is cleaner.

The biggest practical recommendation

For debugging, change the target resolution first:

```
target <- st_as_stars(
  st_bbox(st_transform(trimmed, mollweide_crs)),
  dx = 100000,
```

```
  dy = 100000
)
```

If that suddenly renders quickly, then your original 50 km warp is simply too heavy for rapid preview in Quarto. That is the most likely explanation.

The strongest suspects are:

- `st_warp()` on a global raster cube, [304] [302]
- double `geom_stars()` layers on the warped result, [305] [303]
- and Quarto re-running the whole chunk every preview.

If you want, I can rewrite your code into a faster “draft-preview” version and a final “submission-quality” version.

⁂

How does cache work in quarto?

In Quarto, cache stores the results of expensive code execution so unchanged code chunks do not need to run again every time you render the document. For an R-based `.qmd`, Quarto uses the knitr cache, and cache invalidation is triggered when the chunk source code or cache-related attributes change. [318] [319]

How to enable it

You can turn cache on for the whole document in YAML with `execute: cache: true`, or only for selected chunks with a chunk option like `#| cache: true`. [318] [320] For your lab, the best use is to cache the slow import, warp, and summary chunks rather than every small chunk. [318] [321]

```
---
title: "My Lab"
format: html
execute:
  cache: true
---
```

```
#| label: data-preparation
#| cache: true
```

What happens on rerender

If the cached chunk has not changed, Quarto reuses the saved result instead of recomputing it during render. Changes to prose outside code cells do not invalidate the computation cache, which is why caching is especially helpful when you are editing text around slow code. [319] [318]

Important limitation

Cache is mainly used during document rendering, not as a guarantee for interactive “run cell” behavior inside an editor. That means a chunk may still feel slow when you run it manually in the console or via interactive cell execution, even if it is cached for full Quarto renders. [322] [323]

Refreshing cache

If the code did not change but the underlying input data changed, you may need to force a refresh because Quarto may otherwise keep using the old cached result. You can do that with `quarto render yourfile.qmd --cache-refresh`, and Quarto also supports `--cache` and `--no-cache` command-line options to override normal cache behavior. [318] [319]

For your file, I would add `#| cache: true` to the chunk that builds `warped`, `warped_cropped`, `world_boundary`, and the monthly summaries, because those are the expensive steps most worth reusing across previews. [318] [321]

⁂

This has helped to reduce the long compile times

Great — that means the slow part was most likely repeated recomputation during render, not a fatal problem in your map logic.

What it means

Caching helps because Quarto can reuse the saved result of heavy chunks instead of re-running imports, warping, and raster summaries every preview. That is especially useful in your lab since the reprojection and `stars` operations are much more expensive than the plot styling.

Good practice

Keep caching on for the expensive preparation chunks, especially:

- reading and combining NetCDF files,
- monthly summaries,
- reprojection and warping,
- boundary and helper-layer creation.

You usually do **not** need caching on lightweight plotting-only chunks unless they also build large objects.

One caution

If you change the underlying data, projection settings, or core transformation code, the cached result may no longer reflect what you intend. In that case, refresh or clear the cache so Quarto recomputes those chunks cleanly.

Suggested setup

A good pattern for your file is:

- one cached chunk for data import,
- one cached chunk for summaries,
- one cached chunk for reprojection/prep,
- one uncached chunk for the final plot.

That keeps previews fast while still making it easy to tweak titles, legends, labels, and theme settings.

I noticed this warning while trying to run it in my console,

Warning messages:
1: Removed 251036 rows containing missing values or values outside the scale range (`geom_raster()`).
2: Removed 111084 rows containing missing values or values outside the scale range (`geom_raster()`).

That warning is usually **not fatal**. It means ggplot dropped raster cells that had `NA` values or values it decided were outside the drawable scale range for that layer. In your map, a large number of dropped cells is actually expected to some extent, because land, sea-ice masks, and cropped areas outside the round map are intentionally missing in one layer or another. [328] [329] [330]

Why it happens in your plot

Your first raster layer uses `open_ocean_anom`, which is `NA` everywhere that is not open ocean by design. So all land and sea-ice cells are supposed to be missing in that layer and can trigger removal warnings from the raster geometry. Your projected and cropped raster may also contain blank cells outside the valid world boundary, which adds even more missing values. [330]

The second part of the warning, “or values outside the scale range,” can also appear when a scale has explicit limits. ggplot warnings of this type are known to occur when rows are outside scale limits or are treated as missing after scale handling. [331] [328]

Is it a problem?

Usually, **no**, not if the blank areas are intentional. In your case, you actually want:

- land not colored by the anomaly layer,
- sea ice not colored by the anomaly layer,
- outside-of-map areas blanked,
- only open-ocean anomalies shown in the diverging scale. [330]

So some removal is consistent with the assignment.

One thing to check

You are using:

```
scale_fill_fermenter(...)
```

for `open_ocean_anom`, but that is a binned Brewer scale, while your anomaly field is continuous. The ggplot2 documentation distinguishes `fermenter` for binned colour scales, so switching to `scale_fill_distiller()` is more appropriate here and may reduce confusion in scale handling. [332] [333] [334]

Use:

```
scale_fill_distiller(
  palette = "RdYlBu",
  direction = 1,
  limits = c(-limit, limit),
  oob = scales::squish,
  name = "Anomaly (°C)"
)
```

When to worry

You should worry only if:

- open-ocean anomalies are disappearing where they should be visible,
- the whole map looks patchy unexpectedly,
- or the removed-count is much larger than the number of intentionally blank cells. [330]

If the map itself looks correct, then the warning is mostly informational.

Practical check

A useful check is to count how many cells are intentionally missing in `open_ocean_anom`:

```
sum(is.na(as.vector(warped_cropped$open_ocean_anom)))
```

and compare that with the warning size. If it is large, that supports the idea that most of the dropped rows are expected due to masking.

So the warning is probably normal for your layered raster map, but I would still change `scale_fill_fermenter()` to `scale_fill_distiller()` because that is the more correct scale for continuous anomaly values.

⁂

The map draws the anomaly for open ocean only and collapses sea ice into

a single class. What does the reader gain and lose from these two choices?

The reader gains a cleaner comparison of ocean-temperature anomalies and the large-scale position of sea ice, but loses detail about what is happening **within** the ice-covered zone. In other words, the map becomes easier to read as a two-panel climate story, but less informative as a full sea-ice analysis. [342] [343] [344]

What the reader gains

Showing anomalies only over open ocean prevents hidden or masked values beneath sea ice from influencing the colour scale, so the red-blue anomaly field is easier to interpret and more comparable across the two Decembers. It also makes the anomaly legend speak clearly to one physical question, namely how open-ocean temperatures differ from climatology, instead of mixing that message with ice-covered regions where the surface class is different. [345] [344]

Collapsing sea ice into a single class also reduces visual clutter. Instead of asking the reader to decode many sea-ice concentrations while also reading a diverging anomaly map, the figure communicates a simpler boundary: where sea ice is present at or above the 15% threshold and where open ocean begins. [344]

What the reader loses

The main loss is that the reader can no longer see temperature-anomaly variation in ice-covered waters, even if those anomalies might still matter physically at the ocean surface or near the ice edge. That means the map emphasizes open-ocean thermal patterns at the cost of information about the coupled ocean-ice environment. [343] [344]

By collapsing sea ice into one class, the reader also loses gradation within the ice pack. A cell just above 15% concentration is drawn the same way as a cell with very dense ice cover, so the map no longer shows differences in sea-ice intensity, compactness, or internal structure. [342] [344]

Interpretation trade-off

So the design choice improves readability and panel-to-panel comparison, especially for highlighting El Niño versus La Niña differences in open-ocean anomaly patterns. But it weakens the map as a detailed cryosphere product, because sea ice becomes a boundary category rather than a continuously varying field. [344] [342]

Report-style phrasing

A concise way to write this is: “Restricting anomalies to open ocean makes the diverging colour scale cleaner and avoids interference from ice-covered cells, while collapsing sea ice to a single class makes the map easier to compare between months.” “However, these choices remove information about anomaly patterns beneath ice cover and about variation in sea-ice concentration beyond the 15% threshold.”[344]

⁂

the second image is what was created and the first one is the aim of the labsheet. How do I get them to be more similar? Esepecially, ocean labels seem abit off (2nd image), how should I fix this?

Yes — your map is already close, but compared with the lab target, the main differences are the **ocean label positions/sizing**, the **legend style**, the **equator styling**, and some overall visual restraint. From the two images, your labels look slightly too central and a bit too prominent, while the target places them more carefully inside the basins and keeps them quieter.[353] [354]

Main differences

In your version, the ocean labels are larger, darker, and placed too close to the centers of the panels, so they compete more with the anomaly field. In the lab target, the labels are smaller, lighter, and positioned more naturally inside the ocean basins: Pacific near the central equatorial Pacific, Atlantic farther east and north, and Indian farther west and south.[354] [353]

Your legend is also different: the target uses a more compact, neater legend with simpler discrete-looking color steps, and the equator line is more subtle. The target also leaves more breathing room between the map, labels, and legend.[353] [354]

Why the labels look off

Right now you are placing labels using lon-lat points like:

```
st_point(c(180, 0))
st_point(c(330, 0))
st_point(c(80, -20))
```

Those coordinates are not wrong, but after Mollweide projection they do not land in the most visually balanced positions. Because projected world maps bend and compress space, labels that seem centered in longitude-latitude do not always look centered in the final map.

For Mollweide, better approximate starting positions would be:

- Pacific: around `(200, 5)`
- Atlantic: around `(330, 15)`

- Indian: around `(80, -15)`

That should move the Pacific label a bit east and slightly north, keep the Atlantic higher, and lift the Indian label slightly.

Better label code

Try this:

```
ocean_labels <- st_sf(
  name = c("PACIFIC OCEAN", "ATLANTIC OCEAN", "INDIAN OCEAN"),
  geometry = st_sfc(
    st_point(c(200, 5)),
    st_point(c(330, 15)),
    st_point(c(80, -15)),
    crs = 4326
  )
) |>
  st_transform(mollweide_crs)

ocean_coords <- as.data.frame(st_coordinates(ocean_labels))
ocean_coords$name <- ocean_labels$name
```

Then make the labels smaller and lighter:

```
geom_label(
  data = ocean_coords,
  aes(x = X, y = Y, label = name),
  fill = alpha("white", 0.45),
  color = "grey25",
  size = 2.2,
  fontface = "plain",
  label.size = 0,
  label.padding = unit(0.10, "lines")
)
```

That will make them sit more like annotations instead of focal elements.

Other changes to get closer

To make your figure more like the target:

- Reduce label size and weight.
- Make the equator line thinner and lighter.
- Use a slightly lighter land gray.
- Tighten the legend styling.
- Use `scale_fill_distiller()` instead of `scale_fill_fermenter()` if the lab expects a continuous anomaly field rather than binned categories. [355] [356]

For the equator, try:

```
geom_path(
  data = equator_df, aes(x = X, y = Y),
  colour = "grey70", linetype = "dashed", linewidth = 0.25
)
```

For land:

```
land = "grey75"
```

One important legend point

Your current anomaly legend looks binned into ranges like `+2 to +3`, which is different from the lab target's simpler numeric tick style. That is probably because you are using `scale_fill_fermenter()`, which is for binned colour scales. Switch to:

```
scale_fill_distiller(
  palette = "RdYlBu",
  direction = 1,
  limits = c(-limit, limit),
  oob = scales::squish,
  name = "Anomaly (°C)",
  guide = guide_colorbar(
    frame.colour = "black",
    frame.linewidth = 0.3,
    ticks.colour = "black"
  )
)
```

That should help the legend resemble the target more closely. [356] [355]

Practical approach

The fastest way to improve the label placement is:

1. adjust the three lon-lat points,
2. shrink the labels,
3. lighten the label box,
4. rerender.

A good first trial set is:

```
Pacific  = c(205, 8)
Atlantic = c(330, 18)
Indian   = c(78, -18)
```

Recommended summary changes

Use these as your first fixes:

```
ocean_labels <- st_sf(
  name = c("PACIFIC OCEAN", "ATLANTIC OCEAN", "INDIAN OCEAN"),
  geometry = st_sfc(
    st_point(c(205, 8)),
    st_point(c(330, 18)),
    st_point(c(78, -18)),
    crs = 4326
  )
) |>
  st_transform(mollweide_crs)
```

and

```
geom_label(
  data = ocean_coords,
  aes(x = X, y = Y, label = name),
  fill = alpha("white", 0.45),
  color = "grey25",
  size = 2.2,
  fontface = "plain",
  label.size = 0,
  label.padding = unit(0.10, "lines")
)
```

If you want, I can rewrite your full plotting chunk so it matches the lab target much more closely.

⁂

Could you help me with this error as well? In 3.13, this was the task,

The map you built in Section 3.12 uses a continuous scale: The anomaly fades smoothly from blue through the pale center to red. Build a second version with a binned scale, which sorts the anomaly into a few labeled classes.
Redraw that map, changing only its anomaly fill:

Cut the anomaly into equal-width bounded classes spanning the same −limit to +limit range as the continuous scale—for example whole-degree steps—with an open-ended class beyond each end.
Label the classes in the legend.
Keep every other element identical (projection, surface overlay,
one-column layout, labels, equator, and frame), so the two maps differ
only in how the anomaly is colored

Therefore, I did this,

```
breaks <- seq(-limit, limit, by = 1)

bin_labels <- c(
  paste0("< ", breaks[1], "°C"),
  paste0(head(breaks, -1), " to ", tail(breaks, -1), "°C"),
  paste0("> ", breaks[length(breaks)], "°C")
)

binned_scale_obj <- scale_fill_fermenter(
  palette = "RdYlBu",
  direction = -1,
  breaks = breaks,
  limits = c(-limit, limit),
  oob = scales::squish,
  na.value = NA,
  labels = bin_labels,
  name = "Anomaly (°C)",
  guide = guide_colorsteps(frame.colour = "black", frame.linewidth = 0.3, ticks.colour = "black")
)
```

However, I got this error,
Error in `scale_fill_fermenter()`:
! `breaks` and `labels` must have the same length.
Backtrace:
■

1. └─ggplot2::scale_fill_fermenter(...)
2. └─ggplot2::binned_scale(...)
3. └─ggplot2:::check_breaks_labels(breaks, labels, call = call)
4. └─cli::cli_abort(...)
5. └─rlang::abort(...)

Yes — the error is because your `breaks` vector and your `bin_labels` vector are not the same length, and `scale_fill_fermenter()` checks that strictly. In your code, `breaks <- seq(-limit, limit, by = 1)` gives the **internal break points**, while your labels also include the two open-ended end classes, so you created more labels than breaks. [357] [358]

Why this happens

With a binned color scale, the breaks are the cut points, not the bins themselves. If you want open-ended classes below the first break and above the last break, those are extra bins, so a simple `breaks = ...` plus a fixed label vector often leads to a mismatch.[359] [360] [357]

For example, if `limit = 3`, then:

```
breaks <- seq(-3, 3, by = 1)
```

has length 7, but your labels become:

- `< -3°C`
- `-3 to -2°C`
- `-2 to -1°C`
- `-1 to 0°C`
- `0 to 1°C`
- `1 to 2°C`
- `2 to 3°C`
- `> 3°C`

which is length 8. That is exactly why ggplot throws the error.[357]

Best fix

A simpler and more reliable approach is to let ggplot handle the bins and label the break values, or to pre-bin the variable yourself. Since your lab wants bounded classes plus open-ended extremes, the cleanest option is usually to **pre-bin** `open_ocean_anom` with `cut()` and then map the resulting factor with a manual fill scale. The `guide_coloursteps()` documentation also notes that binned guides work naturally with factor-like bins created outside the scale.[357]

Recommended approach

Create a new categorical attribute first:

```
bin_edges <- c(-Inf, seq(-limit, limit, by = 1), Inf)

bin_labels <- c(
  paste0("< ", -limit),
  paste0(seq(-limit, limit - 1, by = 1), " to ", seq(-limit + 1, limit, by = 1)),
  paste0("> ", limit)
)

dec_binned <- warped_cropped |>
  mutate(
    anom_bin = cut(
      open_ocean_anom,
      breaks = bin_edges,
```

```
      labels = bin_labels,
      include.lowest = TRUE,
      right = TRUE
    )
  )
```

Then plot with a discrete palette instead of trying to force the continuous variable directly through `scale_fill_fermenter()`.[359] [357]

If you want to keep `scale_fill_fermenter()`

Then your `labels` must match the length of `breaks`. In that case, do **not** include both open-ended end-bin labels in the `labels` vector the way you did. But this is awkward, because the open-ended bins are conceptually outside the internal break structure.[361] [357]

So the better answer is: your current strategy is fighting how `scale_fill_fermenter()` expects breaks and labels to work.

Practical takeaway

Your error is caused by:

- `breaks` = cut points,
- `labels` = bin labels including extra end bins,
- lengths not matching.[358] [357]

For this lab, I would strongly recommend pre-binning with `cut()` and then using `scale_fill_manual()` or another discrete scale for the anomaly layer.

Very short fix explanation

If you want one sentence for yourself:

> `scale_fill_fermenter()` expects one label per break, but I supplied one label per bin, including the two extra open-ended end bins.

If you want, I can write the full **Section 3.13** plotting code so it stays identical to your 3.12 map except that the anomaly layer becomes binned.

⁂

This was my code,

```
#| label: binned-scale
#| warning: false

breaks <- seq(-limit, limit, by = 1)
bin_labels <- c(
  paste0("\u2264 ", breaks[1], "°C"),
  paste0(head(breaks, -1), " to ", tail(breaks, -1), "°C"),
```

```
  paste0("> ", breaks[length(breaks)], "°C")
)

binned_mapping <- aes(
  fill = cut(open_ocean_anom, breaks = c(-Inf, breaks, Inf), labels = bin_labels)
)

binned_scale_obj <- scale_fill_fermenter(
  palette = "RdYlBu",
  direction = -1,
  limits = c(-limit, limit),
  oob = scales::squish,
  na.value = NA,
  name = "Anomaly (°C)",
  guide = guide_colorsteps(frame.colour = "black", frame.linewidth = 0.3, ticks.colou
)

map_binned <- make_sst_map(binned_scale_obj, binned_mapping)
map_binned
```

I ran into this error,
Error in `scale_fill_fermenter()`:! Binned scales only support continuous data.

The error makes sense: `scale_fill_fermenter()` is a **binned scale for continuous input**, but your `cut(...)` has already converted `open_ocean_anom` into a **factor**. So ggplot is receiving discrete data, while `scale_fill_fermenter()` expects continuous numeric data that it can bin itself. [372] [373] [374]

Why your current code fails

This line creates categorical values:

```
fill = cut(open_ocean_anom, breaks = c(-Inf, breaks, Inf), labels = bin_labels)
```

`cut()` returns a factor, not a numeric variable. Once you do that, you should use a **discrete** fill scale such as `scale_fill_manual()` or `scale_fill_brewer()`, not `scale_fill_fermenter()`. [375] [372]

Two valid approaches

You have two correct options:

- Keep `open_ocean_anom` numeric, and let `scale_fill_fermenter()` bin it.
- Bin the values yourself with `cut()`, and then use a discrete scale.

Since your lab wants labeled classes like `< -3`, `-3 to -2`, ..., `> 3`, the second approach is usually easier to control.

Best fix for your code

Keep your `cut(...)`, but replace `scale_fill_fermenter()` with `scale_fill_manual()`. For example:

```
breaks <- seq(-limit, limit, by = 1)

bin_labels <- c(
  paste0("≤ ", breaks[^35_1], "°C"),
  paste0(head(breaks, -1), " to ", tail(breaks, -1), "°C"),
  paste0("> ", breaks[length(breaks)], "°C")
)

binned_mapping <- aes(
  fill = cut(
    open_ocean_anom,
    breaks = c(-Inf, breaks, Inf),
    labels = bin_labels,
    include.lowest = TRUE
  )
)

bin_colors <- rev(RColorBrewer::brewer.pal(length(bin_labels), "RdYlBu"))

binned_scale_obj <- scale_fill_manual(
  values = setNames(bin_colors, bin_labels),
  drop = FALSE,
  na.value = NA,
  name = "Anomaly (°C)",
  guide = guide_legend(
    frame.colour = "black",
    keyheight = unit(0.5, "cm")
  )
)

map_binned <- make_sst_map(binned_scale_obj, binned_mapping)
map_binned
```

One important note

If you are **not allowed** to use `RColorBrewer` directly because the lab only listed certain packages, then generate the colors from ggplot's built-in Brewer palette another way, or hardcode a vector of colors. The key point is not the exact color function — it is that once you use `cut()`, the scale must be discrete. [375]

If you want to keep `scale_fill_fermenter()`

Then do **not** use `cut()` inside `aes()`. Instead map the raw numeric field:

```
binned_mapping <- aes(fill = open_ocean_anom)
```

and let the binned scale do the classing. But custom open-ended legend labels are usually more awkward that way. [373] [376] [374]

Short explanation

So the rule is:

- `scale_fill_fermenter()` = numeric input, ggplot bins it. [374]
- `cut(...)` = factor input, you use a discrete scale. [372] [375]

Your code currently mixes both approaches, which is why ggplot complains.

If you want, I can rewrite your `make_sst_map()` call so the 3.13 binned map matches your 3.12 continuous map as closely as possible.

⁂

I had `na.value = NA` in the previous fermenter and it is not present in this brewer, and the intention of that was to ensure that the edges did not have the gray areas, does this also fulfill the same thing?

Not automatically. If your goal is to keep missing anomaly cells **blank** instead of drawing them in a fallback color, then you should explicitly add `na.value = NA` to the discrete scale as well. [387] [388]

For the earlier Brewer/fermenter scale, `na.value` is the argument that controls the color used for missing values, and the documented default is a grey missing-value color unless you override it. So if you switch to a manual discrete scale and leave out `na.value = NA`, you should not assume it will behave identically. [388] [387]

A good version is:

```
binned_scale_obj <- scale_fill_manual(
  values = setNames(bin_colors, bin_labels),
  drop = FALSE,
  na.value = NA,
  na.translate = FALSE,
  name = "Anomaly (°C)"
)
```

Here, `na.value = NA` keeps missing raster cells transparent, and `na.translate = FALSE` prevents missing values from becoming their own legend entry on a discrete scale. [389] [388]

So the short answer is: if the grey edge problem was being prevented by `na.value = NA` before, then you should carry that setting over explicitly to the new discrete scale too. If grey areas still remain after that, they are probably coming from a different layer, not from the binned anomaly fill itself. [387] [388]

What is this error? My teammate had this error after I pushed,

Error:
! OGR error
Backtrace:
■

1. └─global warp_and_plot(equal_earth_crs)
2. ├─sf::st_transform(...)
3. └─sf:::st_transform.sfc(...)

This error means `sf::st_transform()` failed while trying to reproject a geometry object, and the most likely causes are a missing source CRS, an invalid or unsupported target CRS definition, or an environment difference on your teammate's machine. Because the traceback ends in `st_transform.sfc(...)`, the failure is probably happening on one of the geometry objects you build manually, such as the projected boundary, equator line, or ocean-label points, rather than on a plain data frame. [402] [403] [404] [405]

Most likely causes

The first thing to check is whether the object being transformed actually has a CRS before `st_transform()` is called, because `sf` cannot transform an `sfc` object with a missing CRS. In your code, that especially matters for objects created from `st_bbox()`, `st_as_sfc()`, `st_sfc()`, and `st_linestring()`, since those only transform correctly if the lon-lat CRS was attached properly at creation time. [404] [406] [402]

A second common cause is that the target CRS string is not recognized or supported on the other machine. That fits your situation because the error appeared after pushing to a teammate, which often points to differences in GDAL/PROJ or `sf` installation between environments. [407] [403] [405]

In your code

The most suspicious lines are the ones that transform hand-built geometries, such as the world boundary and equator line, because those go through `st_transform.sfc(...)` exactly as shown in the traceback. If `equal_earth_crs` is defined with a custom PROJ string, your machine may accept it while your teammate's GDAL/PROJ setup may not. [408] [405] [404]

If the failing line is something like `st_transform(mollweide_crs)` or `st_transform(equal_earth_crs)` on `world_boundary`, then make sure the bounding box was created with `crs = st_crs(4326)` and not left without CRS metadata. If the failing line is the target-grid creation around `st_transform(trimmed, crs)`, then the issue may instead be that the target CRS definition itself is not available on your teammate's system. [403] [405] [406] [402]

What to check

Ask your teammate to test these in the console and inspect the outputs before the failing transform: [402] [404]

```
st_crs(equal_earth_crs)
st_crs(world_boundary)
st_crs(equator_line)
st_crs(ocean_labels)
```

If any of those show missing CRS information, that is the problem. If the source CRS is present but `st_transform()` still fails only on their machine, then the likely issue is CRS support or formatting in their local spatial libraries. [405] [407] [403] [402]

Safer fix

A practical fix is to rebuild the target CRS inside the script using a standard `st_crs("...")` definition and ensure every manually created geometry is explicitly created with `crs = 4326` before transformation. If the Equal Earth definition is the culprit, using a well-formed authority-backed or fully valid PROJ string consistently across the team is safer than relying on one machine's implicit support. [407] [404] [405]

A concise message for your teammate would be: "The reprojection failed because `st_transform()` could not use the CRS information on one of the geometry objects or could not resolve the Equal Earth CRS on your machine." If you paste the exact line inside `warp_and_plot(equal_earth_crs)` that triggers the error, I can narrow it down further. [405] [402]

⁂

1. https://www.scribd.com/document/893424072/r-Gis-Packages
2. https://cran.r-project.org/web/packages/tidyterra/tidyterra.pdf
3. https://www.youtube.com/watch?v=YOJ_SISl_wg
4. https://www.youtube.com/watch?v=bvlB5yb_uVQ
5. https://kodu.ut.ee/~kmoch/geopython2023/R_08/3D-mapping.html
6. https://www.rayshader.com/
7. https://rdrr.io/github/ngort01/map3d/
8. https://rpkg.net/package/ggspatial
9. https://r-spatial.org/book/07-Introsf.html
10. https://dges.carleton.ca/CUOSGwiki/index.php/R_Studio's_Spatial_Capabilities_going_3D!
11. https://r-geospatial.github.io/r3.html
12. https://journal.r-project.org/articles/RJ-2020-012/RJ-2020-012.pdf
13. https://cran.r-project.org/web/packages/webglobe/vignettes/webglobe.html
14. https://cran.r-project.org/web/packages/sp/index.html
15. https://nrennie.rbind.io/blog/r-packages-for-visualising-spatial-data/

16. https://extreme.kishou.go.jp/tool/itacs-tcc2015/itacs5-tutorial/T01.pdf
17. https://coralreefwatch.noaa.gov/product/5km/tutorial/crw07b_anomaly.php
18. https://www.nhc.noaa.gov/sst/
19. https://www.bom.gov.au/climate/ocean/sst/
20. https://www.wcrp-climate.org/wgsip/documents/Tech-Note-1.pdf
21. https://journals.ametsoc.org/view/journals/phoc/31/8/1520-0485_2001_031_2340_teooco_2.0.co_2.xml
22. https://ascmo.copernicus.org/articles/11/107/2025/ascmo-11-107-2025.html
23. https://www.cpc.ncep.noaa.gov/products/GODAS/ocean_briefing_gif/global_ocean_monitoring_2011_12.pdf
24. https://psl.noaa.gov/people/michael.alexander/Pices.article.pdf
25. https://reanalyses.org/sites/default/files/2018-03/TIRA_Robertson_27Mar18.pdf
26. https://en.wikipedia.org/wiki/Sea_surface_temperature
27. https://www.cpc.ncep.noaa.gov/products/GODAS/ocean_briefing_gif/global_ocean_monitoring_2007_08.pdf
28. https://rainbow.ldeo.columbia.edu/papers/paper2.pdf
29. https://map.meteomadagascar.mg/dochelp/StatTutorial/Climatologies/index.html
30. https://blogs.ubc.ca/colinmahony/2013/12/20/spatio-temporal-standardization/
31. https://www.datawrapper.de/blog/diverging-vs-sequential-color-scales
32. https://blogs.sas.com/content/iml/2014/10/01/colors-for-heat-maps.html
33. https://coralreefwatch.noaa.gov/product/5km/tutorial/crw07b_anomaly.php
34. https://www.bom.gov.au/climate/ocean/sst/
35. https://courses.ems.psu.edu/geog486/node/878
36. https://map.meteomadagascar.mg/dochelp/StatTutorial/Climatologies/index.html
37. https://gmd.copernicus.org/articles/5/245/2012/gmd-5-245-2012.pdf
38. https://chrisholdgraf.com/matplotlib/tutorials/colors/colormaps.html
39. https://betterfigures.org/2015/06/23/picking-a-colour-scale-for-scientific-graphics/
40. https://web.natur.cuni.cz/~langhamr/lectures/vtfg1/mapinfo_2/barvy/colors.html
41. https://www.ipcc.ch/site/assets/uploads/2022/09/IPCC_AR6_WGI_VisualStyleGuide_2022.pdf
42. https://niiv.njitvis.com/assets/publications/2020/tvcg_2020.pdf
43. https://www.bluemarble.ch/files/publications/teuling_bivariate_color_maps.2010.pdf
44. https://r-spatial.r-universe.dev/stars
45. https://rdrr.io/cran/stars/
46. https://cran.r-project.org/web/packages/stars/refman/stars.html
47. https://forum.posit.co/t/failed-installation-of-stars-package-pkg-config-and-gdal-config-not-found/150228
48. https://ftp.cixug.es/CRAN/web/checks/check_results_stars.html
49. https://web-r.org/r-ecosystem/packages/stars/
50. https://packages.guix.gnu.org/packages/r-stars/0.6-8/
51. https://cran.r-project.org/web/packages/stars/vignettes/stars1.html
52. https://packages.ubuntu.com/resolute/r-cran-stars

53. https://www.r-project.org/nosvn/R.check/r-devel-windows-x86_64/stars-00install.html
54. https://rpkg.net/package/stars
55. https://www.rdocumentation.org/packages/stars/versions/0.7-2
56. https://github.com/r-spatial/stars/issues/453
57. https://stackoverflow.com/questions/78069565/installed-package-sf-disappeared-after-failing-to-install-package-stars-and
58. https://forum.posit.co/t/unable-to-install-stars-from-github-non-zero-exit-status/58697
59. https://r-spatial.r-universe.dev/stars
60. https://r-spatial.github.io/sf/
61. https://discourse.jupyter.org/t/error-to-install-sf-package-under-r/32395
62. https://stackoverflow.com/questions/70267445/unable-to-install-sf-on-r-version-4-1-2
63. https://stackoverflow.com/questions/78069565/installed-package-sf-disappeared-after-failing-to-install-package-stars-and
64. https://stackoverflow.com/questions/70183923/unable-to-install-specific-r-packages
65. https://stackoverflow.com/questions/75186554/how-to-solve-error-when-installing-sf-package
66. https://r-spatial.r-universe.dev/stars/doc/manual.html
67. https://cran.r-project.org/web/packages/stars/stars.pdf
68. https://github.com/r-spatial/sf/issues/680
69. https://forum.posit.co/t/error-installing-rgdal-sf-stars-packages-in-r-in-macos-version-12-5-1/145503
70. https://github.com/r-spatial/sf/issues/1393
71. https://stackoverflow.com/questions/67133013/error-package-or-namespace-load-failed-for-sf
72. https://forum.posit.co/t/unable-to-install-stars-from-github-non-zero-exit-status/58697
73. https://forum.posit.co/t/failed-installation-of-stars-package-pkg-config-and-gdal-config-not-found/150228
74. https://www.rdocumentation.org/packages/stars/versions/0.6-8/topics/read_stars
75. https://kodu.ut.ee/~kmoch/geopython2023/R_08/3D-mapping.html
76. https://dges.carleton.ca/CUOSGwiki/index.php/R_Studio's_Spatial_Capabilities_going_3D!
77. https://search.r-project.org/CRAN/refmans/stars/html/read_ncdf.html
78. https://github.com/r-spatial/stars
79. https://tmieno2.github.io/R-as-GIS-for-Economists/read-write-stars.html
80. https://www.rdocumentation.org/packages/stars/versions/0.6-6
81. https://github.com/r-spatial/stars/blob/main/R/read.R
82. https://github.com/loreabad6/chirps-stars
83. https://github.com/r-spatial/stars/issues/646
84. https://r-spatial.github.io/stars/reference/read_ncdf.html
85. https://www.rdocumentation.org/packages/stars/versions/0.7-0/topics/read_ncdf
86. https://r-spatial.r-universe.dev/stars/doc/manual.html
87. https://cran.r-project.org/web/packages/stars/stars.pdf

88. https://r-spatial.github.io/stars/articles/stars1.html
89. https://gis.stackexchange.com/questions/357487/read-netcdf-file-with-read-stars-in-r
90. https://r-spatial.github.io/stars/articles/stars8.html
91. https://www.rdocumentation.org/packages/stars/versions/0.7-0/topics/read_stars
92. https://search.r-project.org/CRAN/refmans/stars/html/read_ncdf.html
93. https://cran.r-project.org/web/packages/stars/stars.pdf
94. https://kodu.ut.ee/~kmoch/geopython2023/R_08/3D-mapping.html
95. https://dges.carleton.ca/CUOSGwiki/index.php/R_Studio's_Spatial_Capabilities_going_3D!
96. https://gis.stackexchange.com/questions/357487/read-netcdf-file-with-read-stars-in-r
97. https://github.com/loreabad6/chirps-stars
98. https://www.rdocumentation.org/packages/stars/versions/0.7-0/topics/read_ncdf
99. https://www.rdocumentation.org/packages/stars/versions/0.6-6
100. https://cran.r-project.org/web/packages/stars/vignettes/stars8.html
101. https://cran.r-project.org/web/packages/stars/vignettes/stars1.html
102. https://rdrr.io/cran/stars/src/R/read.R
103. https://tmieno2.github.io/R-as-GIS-for-Economists/read-write-stars.html
104. https://github.com/r-spatial/stars/blob/main/R/read.R
105. https://stackoverflow.com/questions/75305037/dimension-problem-while-reading-netcdf4-files-in-stars-r-package
106. https://github.com/r-spatial/stars
107. https://r-spatial.github.io/stars/reference/read_ncdf.html
108. https://search.r-project.org/CRAN/refmans/stars/html/read_stars.html
109. https://search.r-project.org/CRAN/refmans/stars/help/read_stars.html
110. https://www.rdocumentation.org/packages/stars/versions/0.1-1/topics/read_stars
111. https://www.rdocumentation.org/packages/stars/versions/0.7-2/topics/read_stars
112. https://github.com/r-spatial/stars/blob/main/R/read.R
113. https://cran.r-project.org/web/packages/stars/stars.pdf
114. https://r-spatial.r-universe.dev/stars/doc/manual.html
115. https://rdrr.io/cran/stars/src/R/read.R
116. https://www.rdocumentation.org/packages/stars/versions/0.7-0/topics/read_stars
117. https://cran.r-project.org/web/packages/stars/vignettes/stars1.html
118. https://www.rdocumentation.org/packages/stars/versions/0.7-0/topics/read_stars
119. https://cran.r-project.org/web/packages/stars/stars.pdf
120. https://search.r-project.org/CRAN/refmans/stars/html/read_stars.html
121. https://search.r-project.org/CRAN/refmans/stars/help/read_stars.html
122. image.jpg
123. https://search.r-project.org/CRAN/refmans/stars/html/st_apply.html
124. https://r-spatial.github.io/stars/reference/st_apply.html

125. https://github.com/r-spatial/stars/issues/635
126. https://www.rdocumentation.org/packages/stars/versions/0.6-6
127. https://githubhelp.com/r-spatial/stars
128. https://github.com/r-spatial/stars/issues/176
129. https://rdrr.io/cran/stars/src/R/ops.R
130. https://r-spatial.r-universe.dev/stars/doc/manual.html
131. https://search.r-project.org/CRAN/refmans/stars/html/st_downsample.html
132. https://github.com/r-spatial/stars/issues/430
133. https://cran.r-project.org/web/packages/stars/stars.pdf
134. https://stackoverflow.com/questions/70959666/output-of-st-apply-needs-to-be-reshuffled
135. https://r-spatial.org/python/07-Introsf.html
136. https://gis.stackexchange.com/questions/432367/how-to-apply-mean-min-max-merge-functions-on-a-vector-layer-to-create-a-stars-ra
137. https://gis.stackexchange.com/questions/389775/compute-mean-of-two-stars-objects-in-r
138. image.jpg
139. https://search.r-project.org/CRAN/refmans/stars/html/st_apply.html
140. https://r-spatial.github.io/stars/reference/st_apply.html
141. image.jpg
142. https://search.r-project.org/CRAN/refmans/stars/html/st_apply.html
143. https://r-spatial.github.io/stars/reference/st_apply.html
144. https://cran.r-project.org/web/packages/stars/stars.pdf
145. https://www.rdocumentation.org/packages/stars/versions/0.7-0/topics/read_stars
146. image.jpg
147. image.jpg
148. https://pmc.ncbi.nlm.nih.gov/articles/PMC9441380/
149. https://www.mdpi.com/2072-4292/14/23/6143
150. https://www.sciencedirect.com/science/article/pii/S0034425721005009
151. https://ceur-ws.org/Vol-3702/paper24.pdf
152. https://sites.ualberta.ca/~ahamann/teaching/renr690/seminars/TerraAdvanced.pdf
153. https://rspatial.org/rs/5-supclassification.html
154. https://www.ctc-n.org/system/files/dossier/3b/Output 3.9 Manual for Forest and Land-use Classification.pdf
155. https://ntrs.nasa.gov/api/citations/20030093587/downloads/20030093587.pdf
156. https://www.hec.usace.army.mil/confluence/rasdocs/rmum/6.6/land-classification-layers
157. https://landy.mapbiomas.id/en/glossary
158. https://www.rdocumentation.org/packages/stars/versions/0.7-2/topics/read_stars
159. https://ggplot2.tidyverse.org/reference/scale_brewer.html
160. https://github.com/tidyverse/ggplot2/issues/866

161. image.jpg
162. https://www.rdocumentation.org/packages/ggplot2/versions/4.0.2/topics/scale_colour_brewer
163. https://ggplot2-book.org/scales-guides.html
164. https://stackoverflow.com/questions/39391872/reverse-color-palette-with-legend-limits-ggplot2
165. https://groups.google.com/g/ggplot2/c/wKfOaSxcDCI
166. https://stackoverflow.com/questions/77588025/r-scales-function-oob-squish-will-not-squish-limits-from-0-to-1/77588627
167. https://stackoverflow.com/questions/23653951/r-ggplot2-how-do-i-specify-out-of-bounds-values-colour
168. https://rstudio.github.io/cheatsheets/html/data-visualization.html
169. https://datavizs24.classes.andrewheiss.com/resource/colors.html
170. https://bookdown.dongzhuoer.com/hadley/ggplot2-book/numeric
171. https://ggplot2-book.org/scales-colour.html
172. https://rdrr.io/cran/ggplot2/src/R/scale-brewer.R
173. http://r-statistics.co/R-Color-Theory-ggplot2.html
174. https://blog.csdn.net/qq_38607066/article/details/103865397
175. https://ggplot2.tidyverse.org/reference/scale_brewer.html
176. https://www.rdocumentation.org/packages/ggplot2/versions/4.0.2/topics/scale_colour_brewer
177. https://r-spatial.github.io/stars/reference/st_warp.html
178. https://rdrr.io/cran/stars/man/st_warp.html
179. https://www.riinu.me/2022/02/world-map-ggplot2/
180. https://ggplot2-book.org/coord.html
181. image.jpg
182. https://www.youtube.com/watch?v=hOSVctFvXAk
183. https://github.com/Permian-Global-Research/ezwarp
184. https://developmentseed.org/warp-resample-profiling/
185. https://cran.r-project.org/web/packages/countryatlas/vignettes/sf-and-projections.html
186. https://www.youtube.com/watch?v=HkBXa6Bfhmw
187. https://postgis.net/docs/RT_ST_Transform.html
188. https://groups.google.com/g/ggplot2/c/haNa6PNWn6M
189. https://geobgu.xyz/r-2020/geometric-operations-with-rasters.html
190. https://github.com/paleolimbot/ggspatial/blob/master/R/layer-spatial-stars.R
191. https://github.com/astropy/reproject
192. https://cran.rstudio.com/web/packages/stars/vignettes/stars5.html
193. https://www.rdocumentation.org/packages/sf/versions/1.1-1/topics/st_bbox
194. https://r-tmap.github.io/tmap/articles/41_advanced_crs.html
195. https://github.com/r-spatial/sf/issues/1169

196. https://gis.stackexchange.com/questions/485177/create-a-sf-from-bounding-box-coordinates-and-compare-to-another-sf-in-r
197. https://r-spatial.github.io/sf/reference/st_transform.html
198. https://bin-ye.com/post/2019/07/01/geocomputation-with-r-06/
199. https://www.youtube.com/watch?v=a5eJwrt7Bo8
200. https://stackoverflow.com/questions/55050684/transform-result-of-st-bbox-to-other-crs
201. https://github.com/r-spatial/sf/issues/1481
202. https://cran.r-project.org/web/packages/sf/sf.pdf
203. https://github.com/r-spatial/sf/issues/587
204. https://search.r-project.org/CRAN/refmans/CDSE/html/Point2Bbox.html
205. https://www.rdocumentation.org/packages/sf/versions/0.4-2/topics/st_bbox
206. https://search.r-project.org/CRAN/refmans/pgirmess/html/bbox2sf.html
207. https://docs.ropensci.org/terrainr/reference/addbuff.html
208. https://github.com/r-spatial/sf/issues/2111
209. https://github.com/r-spatial/sf/issues/1078
210. image.jpg
211. https://rstudio.github.io/cheatsheets/sf.pdf
212. https://postgis.net/docs/manual-3.5/it/ST_Segmentize.html
213. https://r-spatial.github.io/sf/reference/st_transform.html
214. https://www.rdocumentation.org/packages/sf/versions/1.1-1/topics/st_bbox
215. https://stackoverflow.com/questions/24496143/sample-points-on-polygon-edge-in-r
216. https://r-spatial.github.io/stars/reference/st_warp.html
217. https://r-spatial.github.io/sf/articles/sf1.html
218. https://www.rdocumentation.org/packages/nngeo/versions/0.4.8/topics/st_segments
219. https://paleolimbot.github.io/ggspatial/reference/geom_spatial_segment.html
220. https://stackoverflow.com/questions/68359161/from-points-to-buffer-passing-through-a-smoothed-line-using-sf-package
221. https://rstudio-pubs-static.s3.amazonaws.com/488297_b0b54ede7c844e639e47dff70c028b37.html
222. https://stat.ethz.ch/pipermail/r-sig-geo/2016-December/025238.html
223. https://r-spatial.github.io/sf/reference/geos_unary.html
224. https://www.rdocumentation.org/packages/sf/versions/1.0-23/topics/st_sample
225. https://gis.stackexchange.com/questions/264734/breaking-long-line-segments-into-shorter-ones-in-r-using-sf
226. https://stat.ethz.ch/pipermail/r-help/2009-May/391833.html
227. https://sape.inf.usi.ch/quick-reference/ggplot2/coord.html
228. https://is.muni.cz/el/1456/podzim2017/BPE_AVED/um/kniha/souradnicove-systemy.html
229. image.jpg
230. https://stackoverflow.com/questions/31939838/ggplot2-aspect-ratio-overpowers-coord-equal-or-coord-fixed

231. https://groups.google.com/g/ggplot2/c/zaM6cklNd3E
232. https://stackoverflow.com/questions/12443620/why-coord-equal-doesnt-work-the-way-it-should
233. https://www.cnblogs.com/wkslearner/p/5718928.html
234. https://github.com/tidyverse/ggplot2/issues/153
235. https://samos-r.netlify.app/intro_r/13-mapping
236. https://stat.ethz.ch/pipermail/r-help/2008-January/151355.html
237. https://icaci.org/files/documents/factsheets/world_map_projections_2026-03-16.pdf
238. https://www.esri.com/about/newsroom/arcuser/equal-earth
239. https://www.earthguessr.com/blog/what-is-the-mollweide-projection
240. https://desktop.arcgis.com/en/arcmap/latest/map/projections/mollweide.htm
241. https://en.wikipedia.org/wiki/Equal_Earth_projection
242. https://en.wikipedia.org/wiki/Mollweide_projection
243. https://www.mathworks.com/help/map/mollweid.html
244. image.jpg
245. https://desktop.arcgis.com/es/arcmap/latest/map/projections/mollweide.htm
246. https://en.wikipedia.org/wiki/List_of_map_projections
247. https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=42082b3eefac56c8b0bcaca09479810509d4cf3d
248. https://map-projections.net/compare.php?p1=eckert-4&p2=mollweide&w=0
249. https://truesize.world/blog/map-projections-at-a-glance/
250. https://www.reddit.com/r/geography/comments/1h88s46/is_the_2018_equal_earth_projection_the_most/
251. https://master.grad.hr/hdgg/kog_stranica/kog15/2Lapaine-KoG15.pdf
252. https://docs.os.uk/more-than-maps/geographic-data-visualisation/guide-to-cartography/projections
253. https://www.earthguessr.com/blog/what-is-the-mollweide-projection
254. https://desktop.arcgis.com/en/arcmap/latest/map/projections/mollweide.htm
255. https://truesize.world/blog/map-projections-at-a-glance/
256. https://desktop.arcgis.com/en/arcmap/10.3/guide-books/map-projections/mollweide.htm
257. https://en.wikipedia.org/wiki/Mollweide_projection
258. https://de.wikipedia.org/wiki/Mollweide-Projektion
259. https://www.studocu.vn/vn/document/truong-dai-hoc-su-pham-ha-noi-2/dia-ly-dai-cuong/phep-chieu-mollweide-dac-diem-va-uu-nhuoc-diem/141863909
260. https://icaci.org/files/documents/factsheets/world_map_projections_2026-03-16.pdf
261. https://www.esri.com/about/newsroom/arcuser/equal-earth
262. https://zattapo.com/mollweide_projection/
263. https://map-projections.net/compare.php?p1
264. https://map-projections.net/compare.php?p1=eckert-4&p2=mollweide&w=0
265. https://master.grad.hr/hdgg/kog_stranica/kog15/2Lapaine-KoG15.pdf

266. https://www.gauthmath.com/solution/1813103776960582/14-Explain-the-strengths-and-weaknesses-of-the-Mollweide-Projection-How-does-it-
267. https://www.studycountry.com/wiki/what-are-the-cons-of-mollweide-projection
268. https://quizlet.com/study-guides/the-advantages-vs-disadvantages-of-map-projections-a5ecb56f-d7de-4a83-95be-32cbfc6fa398
269. https://icaci.org/files/documents/factsheets/world_map_projections_2026-03-16.pdf
270. https://desktop.arcgis.com/en/arcmap/latest/map/projections/mollweide.htm
271. https://www.esri.com/about/newsroom/arcuser/equal-earth
272. https://en.wikipedia.org/wiki/Equal_Earth_projection
273. https://rdrr.io/cran/stars/man/st_dimensions.html
274. https://r-spatial.github.io/stars/reference/st_dimensions.html
275. https://rdrr.io/cran/stars/src/R/dimensions.R
276. https://github.com/r-spatial/stars/issues/100
277. https://ggplot2.tidyverse.org/reference/scale_brewer.html
278. https://www.rdocumentation.org/packages/ggplot2/versions/4.0.2/topics/scale_colour_brewer
279. image.jpg
280. https://r-spatial.r-universe.dev/stars/doc/manual.html
281. https://github.com/r-spatial/stars/blob/main/R/extract.R
282. https://www.rdocumentation.org/packages/stars/versions/0.1-1/topics/st_dimensions
283. https://r-spatial.github.io/stars/articles/stars6.html
284. https://r-spatial.github.io/stars/articles/stars1.html
285. https://cran.r-project.org/web/packages/starsTileServer/vignettes/using_functions.html
286. https://github.com/r-spatial/stars/blob/main/R/stars.R
287. https://github.com/r-spatial/stars/issues/700
288. https://github.com/r-spatial/stars/issues/688
289. https://cran.r-project.org/web/packages/stars/stars.pdf
290. https://www.rdocumentation.org/packages/stars/versions/0.5-2/topics/st_dimensions
291. https://ggplot2.tidyverse.org/reference/guide_legend.html
292. https://www.rdocumentation.org/packages/ggplot2/versions/3.1.0/topics/guide_colourbar
293. image.jpg
294. https://ggplot2-book.org/themes.html
295. https://r-statistics.co/Complete-Ggplot2-Tutorial-Part2-Customizing-Theme-With-R-Code.html
296. https://r-statistics.co/ggplot2-Legends-in-R.html
297. https://cran.r-project.org/web/packages/ggguides/ggguides.pdf
298. https://tidyverse.github.io/ggplot2-docs/reference/guides.html
299. https://tidyverse.org/blog/2024/02/ggplot2-3-5-0-legends/
300. https://ggplot2.tidyverse.org/reference/Guide.html

301. https://aosmith.rbind.io/2020/07/09/ggplot2-override-aes/
302. https://gis.stackexchange.com/questions/414621/reprojected-data-from-stars-r-package-takes-forever-with-geom-stars-and-ggplot2
303. https://r-spatial.github.io/stars/reference/geom_stars.html
304. https://r-spatial.github.io/stars/reference/st_warp.html
305. https://github.com/r-spatial/stars/issues/503
306. https://ggplot2-book.org/scales-colour.html
307. https://bookdown.dongzhuoer.com/hadley/ggplot2-book/binned-colour-scales
308. https://ggplot2.tidyverse.org/reference/scale_brewer.html
309. https://stackoverflow.com/questions/65627153/specify-bin-colours-in-binned-colour-fill-scales
310. https://github.com/r-spatial/stars/issues/21
311. https://stackoverflow.com/questions/64013935/custom-color-palette-for-scale-fill-fermenter
312. https://rdrr.io/cran/colorspace/man/scale_colour_binned_sequential.html
313. https://rdrr.io/cran/ggplot2/src/R/scale-brewer.R
314. https://ggplot2-book.org/scales-guides.html
315. https://stackoverflow.com/questions/70942728/understanding-color-scales-in-ggplot2
316. https://ggplot2.tidyverse.org/reference/index.html
317. https://bookdown.dongzhuoer.com/hadley/ggplot2-book/colour-continuous
318. https://quarto.org/docs/computations/caching.html
319. https://quarto.org/docs/projects/code-execution.html
320. https://www.datanovia.com/guide/tools/quarto/rendering.html
321. https://biostats-r.github.io/biostats/quarto/09-other-topics.html
322. https://github.com/quarto-dev/quarto-cli/discussions/3697
323. https://github.com/orgs/quarto-dev/discussions/3697
324. https://quarto.org/docs/get-started/computations/rstudio.html
325. https://github.com/orgs/quarto-dev/discussions/7491
326. https://www.datanovia.com/es/guide/tools/quarto/rendering.html
327. https://quarto.org/docs/computations/r.html
328. https://stackoverflow.com/questions/32505298/explain-ggplot2-warning-removed-k-rows-containing-missing-values
329. https://github.com/tidyverse/ggplot2/issues/6338
330. image.jpg
331. https://groups.google.com/g/ggplot2-dev/c/t-H0ArYDX8k
332. https://ggplot2.tidyverse.org/reference/scale_brewer.html
333. https://ggplot2-book.org/scales-colour.html
334. https://bookdown.dongzhuoer.com/hadley/ggplot2-book/binned-colour-scales
335. https://stackoverflow.com/questions/71624619/ggplot-warnings-removed-xx-rows-containing-missing-values-geom-col

336. https://github.com/tidyverse/ggplot2/issues/2234
337. https://stackoverflow.com/questions/77023161/removed-xx-rows-containing-missing-values-lower-value-than-expected
338. https://github.com/tidyverse/ggplot2/issues/4083
339. https://github.com/tidyverse/ggplot2/issues/4200
340. https://www.tutorialspoint.com/article/how-to-deal-with-warning-removed-n-rows-containing-missing-values-while-using-ggplot2-in-r
341. https://www.geeksforgeeks.org/r-language/how-to-fix-values-not-appearing-in-ggplot-plot-in-r/
342. https://globalocean.noaa.gov/the-ocean/arctic-sea-ice/
343. https://www.frontiersin.org/journals/marine-science/articles/10.3389/fmars.2021.642372/full
344. image.jpg
345. https://myoceanhealth.marine.copernicus.eu/en?i=eyJzc3QiOnsidmFsdWUiOiJTZWEgU3VyZmFjZSBUZW1wZXJhdHVyZSIsImtleSI6InNzdCIsInZpc2libGUiOnRydWUsInNlbGVjdGVkWWVhcnMiOlsiMjAyNiIsIjIwMjUiLCJtZWFuIiwiMjAyNCJdLCJyb3RhdGUiOmZhbHNlLCJzaG93VW5jZXJ0YWludHkiOnRydWV9LCJtaHciOnsidmFsdWUiOiJNYXJpbmUgSGVhdCBXYXZlcyIsImtleSI6Im1odyIsInZpc2libGUiOmZhbHNlLCJzZWxlY3RlZFllYXJzIjpbIjIwMjYiLCIyMDI1Il0sInJvdGF0ZSI6ZmFsc2V9LCJzbCI6eyJ2YWx1ZSI6IlNlYSBMZXZlbCIsImtleSI6InNsIiwidmlzaWJsZSI6ZmFsc2UsInNlbGVjdGVkWWVhcnMiOlsiMjAyNiIsIjIwMjUiLCIyMDI0Il0sInJvdGF0ZSI6ZmFsc2V9LCJvYSI6eyJ2YWx1ZSI6Ik9jZWFuIEFjaWRpZmljYXRpb24iLCJrZXkiOiJvYSIsInZpc2libGUiOmZhbHNlLCJzZWxlY3RlZFllYXJzIjpbIjIwMjYiLCIyMDI1Il0sInJvdGF0ZSI6ZmFsc2UsInNob3dVbmNlcnRhaW50eSI6dHJ1ZX0sImFyY3NpIjp7InZhbHVlIjoiQXJjdGljIFNlYSBJY2UgRXh0ZW50Iiwia2V5IjoiYXJjc2kiLCJ2aXNpYmxlIjpmYWxzZSwic2VsZWN0ZWRZZWFycyI6WyIyMDI2IiwiMjAyNSIsIm1lYW4iXSwicm90YXRlIjpmYWxzZX0sImFudGFzaSI6eyJ2YWx1ZSI6IkFudGFyY3RpYyBTZWEgSWNlIEV4dGVudCIsImtleSI6ImFudGFzaSIsInZpc2libGUiOmZhbHNlLCJzZWxlY3RlZFllYXJzIjpbIjIwMjYiLCIyMDI1IiwibWVhbiJdLCJyb3RhdGUiOmZhbHNlfX0=&t=2024-06-01
346. https://essd.copernicus.org/preprints/essd-2025-653/essd-2025-653.pdf
347. https://www.reddit.com/r/dataisbeautiful/comments/15khok1/oc_chart_showing_the_antarctic_seaice_extent/
348. https://eprints.soton.ac.uk/412805/1/manuscript_arctic.pdf
349. https://www.nature.com/articles/s41597-022-01166-z
350. https://www.pmel.noaa.gov/arctic-zone/future/bib/Vihma_accepted_in_Surv_Geophys.pdf
351. https://www.mdpi.com/2072-4292/15/23/5563
352. https://oceanvisions.org/wp-content/uploads/2026/05/Arctic-sea-ice-and-climate-stability.pdf
353. image.jpg
354. image-2.jpg
355. https://ggplot2.tidyverse.org/reference/scale_brewer.html
356. https://ggplot2-book.org/scales-colour.html
357. https://ggplot2.tidyverse.org/reference/guide_coloursteps.html
358. https://ggplot2.tidyverse.org/reference/Scale.html
359. https://mpn.metworx.com/packages/ggplot2/3.3.6/reference/scale_steps.html
360. https://stackoverflow.com/questions/68679342/labelling-with-scale-fill-stepsn-breaks-and-labels-are-different-lengths
361. https://stackoverflow.com/questions/71976832/custom-labels-for-limit-and-break-values-when-using-scale-color-binned-in-r-ggpl
362. https://ggplot2-book.org/scales-guides.html

363. https://plotly.com/ggplot2/Guides axes and legends/guide_coloursteps/
364. https://stackoverflow.com/questions/71716811/add-label-names-on-the-color-bar-manually-using-ggplot2-package
365. https://bookdown.org/Maxine/ggplot2-maps/posts/2019-11-27-using-scales-package-to-modify-ggplot2-scale/
366. https://stackoverflow.com/questions/67357589/position-the-labels-at-the-middle-of-the-bin-in-a-binned-step-colorbar-guide
367. https://flashsherlock.github.io/2021/03/30/ggplot2-scales/
368. https://ggplot2-book.org/scales-colour.html
369. https://ggplot2.tidyverse.org/reference/scale_colour_continuous.html
370. https://github.com/tidyverse/ggplot2/issues/4831
371. https://stackoverflow.com/questions/65947347/r-how-to-manually-set-binned-colour-scale-in-ggplot
372. https://stackoverflow.com/questions/73954503/error-binned-scales-only-support-continuous-data-ggplot
373. https://ggplot2-book.org/scales-colour.html
374. https://bookdown.dongzhuoer.com/hadley/ggplot2-book/binned-colour-scales
375. https://bookdown.dongzhuoer.com/hadley/ggplot2-book/colour-discrete
376. https://ggplot2.tidyverse.org/reference/scale_colour_continuous.html
377. https://rdrr.io/cran/ggplot2/src/R/scale-colour.R
378. https://search.r-project.org/CRAN/refmans/ggplot2/help/scale_colour_continuous.html
379. https://stackoverflow.com/questions/73407093/error-scale-for-fill-is-already-present-ggplot2
380. https://github.com/tidyverse/ggplot2/issues/5996
381. https://groups.google.com/g/ggplot2/c/bN5ZtwILYxY
382. https://stackoverflow.com/questions/74734799/scale-fill-manual-factor-levels-colours-or-order-ignored
383. https://stackoverflow.com/questions/70942728/understanding-color-scales-in-ggplot2
384. https://bookdown.dongzhuoer.com/hadley/ggplot2-book/colour-continuous
385. https://qiita.com/swathci/items/00b4a847843a33345887
386. https://stackoverflow.com/questions/60853082/function-scale-fill-fermenter-not-working
387. https://ggplot2.tidyverse.org/reference/scale_brewer.html
388. https://stackoverflow.com/questions/45144630/scale-fill-manual-define-color-for-na-values/45147172
389. https://ggplot2.tidyverse.org/reference/scale_manual.html
390. https://ggplot2-book.org/scales-colour.html
391. https://www.bioprocessintl.com/bioreactors/lessons-in-bioreactor-scale-up-part-1-mdash-exploring-introductory-principles
392. https://github.com/tidyverse/ggplot2/issues/5286
393. https://rdrr.io/cran/ggplot2/src/R/scale-brewer.R
394. https://github.com/tidyverse/ggplot2/issues/5929
395. https://github.com/tidyverse/ggplot2/issues/5298
396. https://github.com/tidyverse/ggplot2/issues/4567
397. https://stackoverflow.com/questions/63382618/ggplot-suppresses-na-colouring

398. https://sjspielman.github.io/introverse/articles/color_fill_scales.html
399. https://rstudio.github.io/cheatsheets/html/data-visualization.html
400. https://ggplot2-book.org/scales-guides.html
401. https://flashsherlock.github.io/2021/03/30/ggplot2-scales/
402. https://stackoverflow.com/questions/70989790/resolving-this-error-cannot-transform-sfc-object-with-missing-crs
403. https://github.com/r-spatial/sf/issues/1419
404. https://r-spatial.github.io/sf/reference/st_transform.html
405. https://forum.posit.co/t/error-in-cpl-transform-x-crs-aoi-pipeline-reverse-desired-accuracy-crs-not-found-is-it-missing/148767
406. https://www.rdocumentation.org/packages/sf/versions/1.1-1/topics/st_bbox
407. https://gis.stackexchange.com/questions/387041/sfst-transform-gives-crs-not-found-error
408. https://github.com/r-spatial/sf/issues/509
409. https://github.com/r-spatial/sf/issues/1634
410. https://stackoverflow.com/questions/76008637/cannot-transform-sfc-object-with-missing-crs-but-st-crs-shows-object-does-have
411. https://stackoverflow.com/questions/73905232/error-in-st-transform-sfcst-geometryx-crs-cannot-transform-sfc-objec
412. https://github.com/r-spatial/sf/issues/1147
413. https://stackoverflow.com/questions/67212607/error-ogr-unsupported-geometry-type-when-using-st-as-sfc-from-sf-package-in-r
414. https://gis.stackexchange.com/questions/316472/ogr-not-enough-data-error-when-using-st-transform-in-r
415. https://github.com/r-spatial/sf/issues/1934
416. https://cloud.tencent.com/developer/ask/sof/106672517/answer/132092694
417. https://forum.posit.co/t/joining-of-sf-objects-as-dataframe-fails-ogr-not-enough-data-gdal-wkb-is-too-small/28441